

Lecture 23

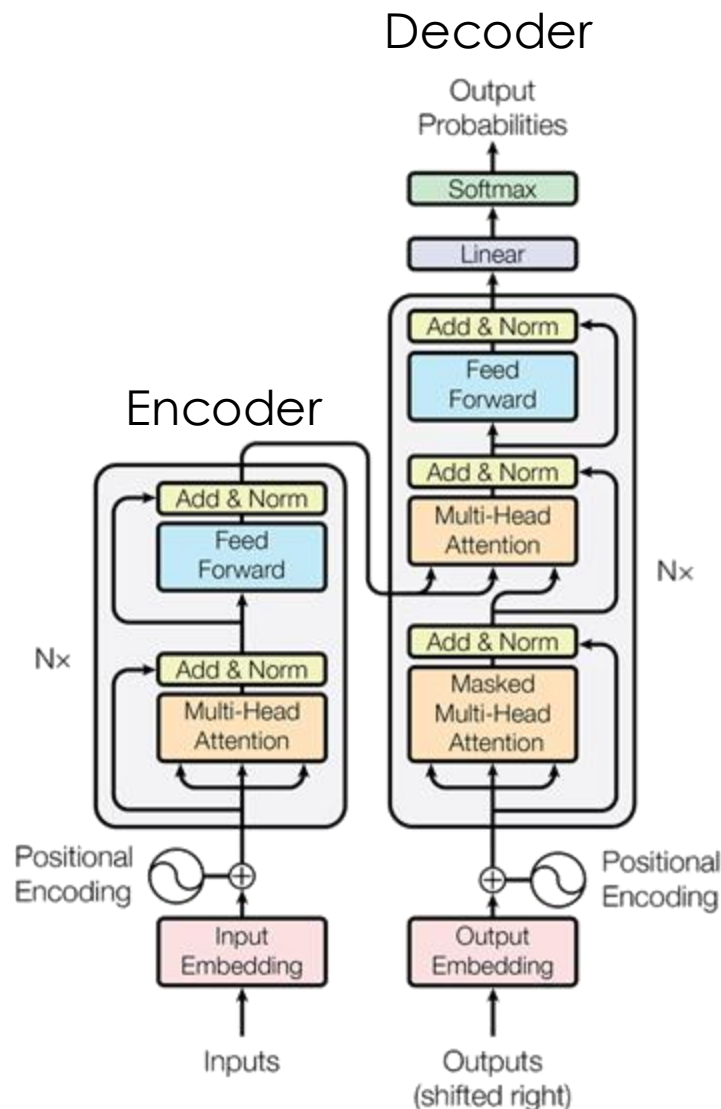
LLM Training and Applications

Pre-training, post-training, and using LLMs.

EECS 189/289, Spring 2026 @ UC Berkeley

Recap – Causal Language Modeling

The Encoder-Decoder Transformer



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

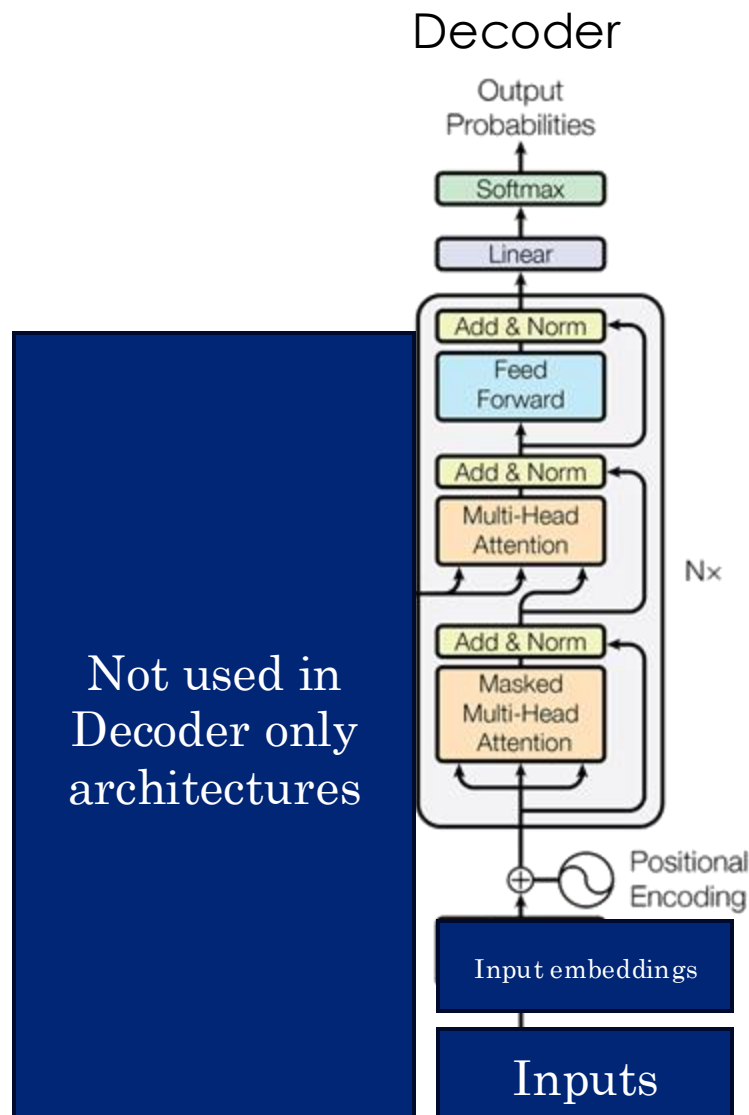
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

The Encoder-Decoder Transformer



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

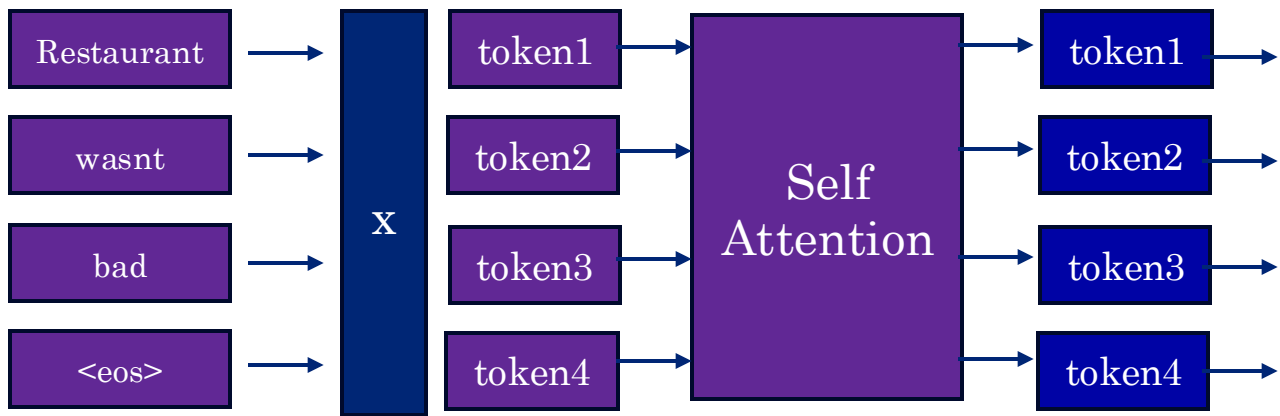
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

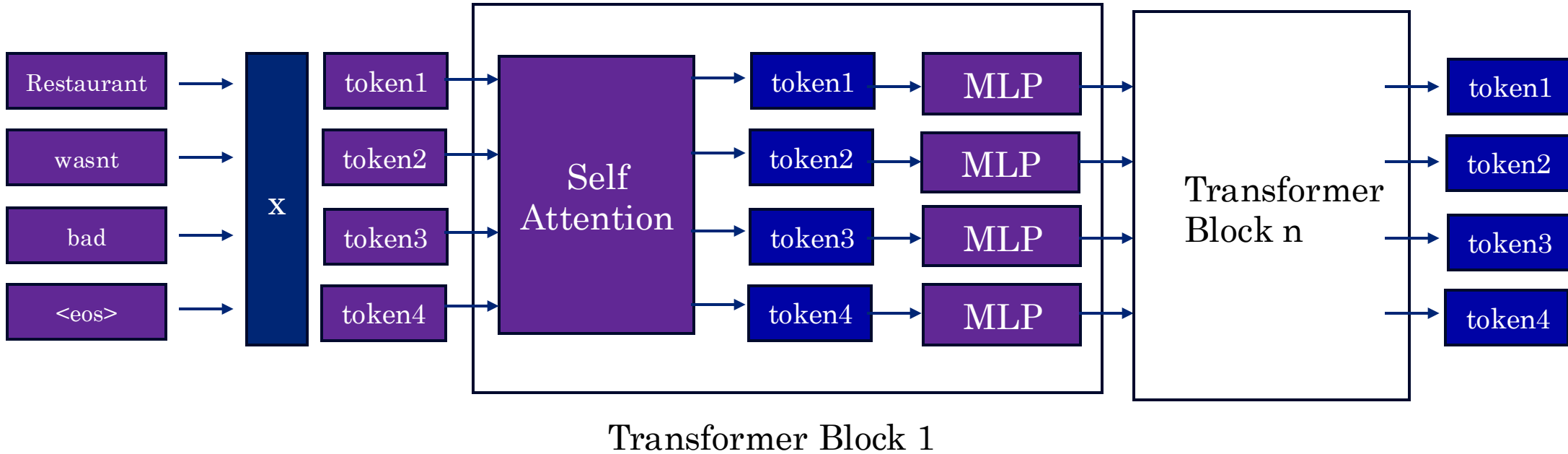
[‡]Work performed while at Google Research.

How do we use a transformer to classify?

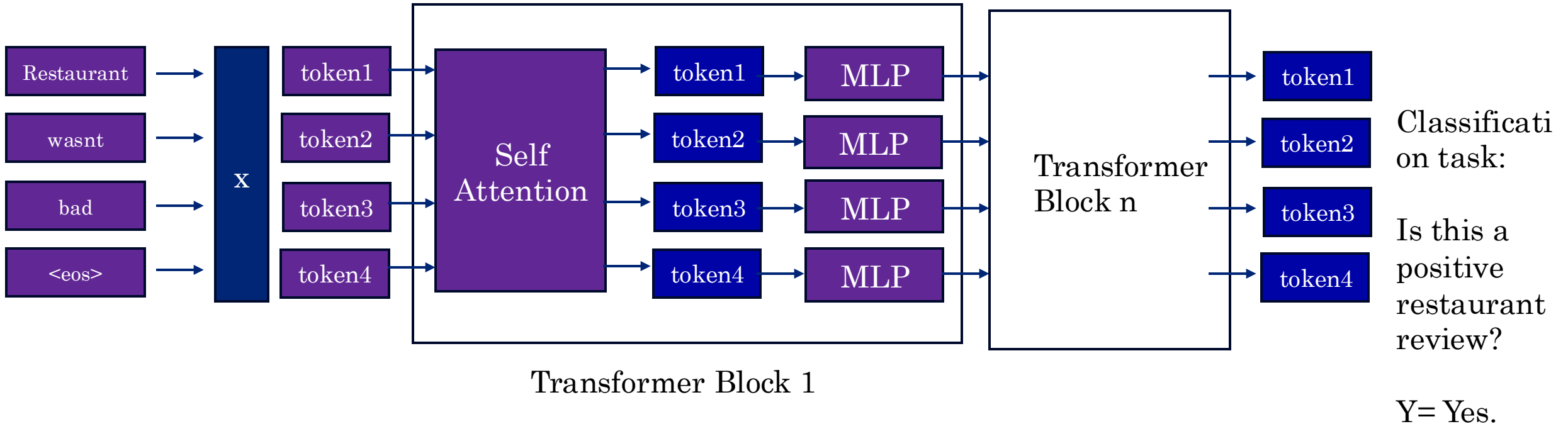


Transformer Block 1

How do we use a transformer to classify?



How do we use a transformer to classify?



GPT-3 175B DaVinci

$D = 12k$, $N = 2048$ tokens.

96 transformer blocks.

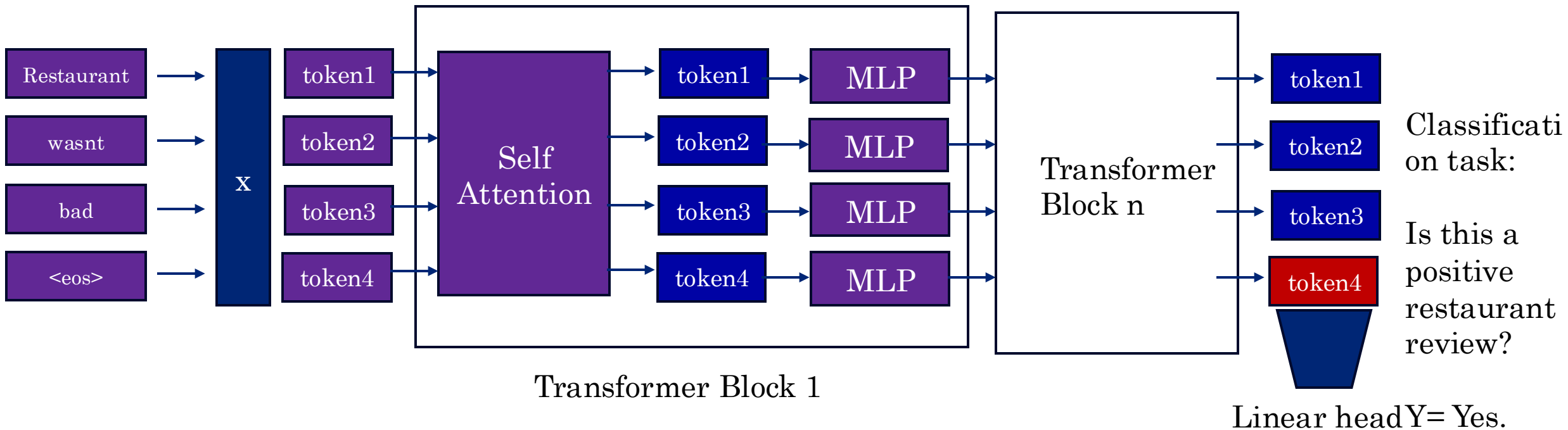
W_q, W_k, W_v is $d \times d$ so $4 \cdot d^2 = 603M$

MLP parameters: $d \rightarrow 4d \rightarrow d$, so $2 \cdot d \cdot 4d = 8d^2 = 1.21B$

Total across 96 layers: Attention: 58B, MLP: 116B

Most of the Transformer parameters are in the MLPs not attention.

How do we use a transformer to classify?



Now say I want to do a linear classification problem.
e.g. is this a positive or negative restaurant review?

I want a representation of the whole input x.

Use the final representation of the last token.

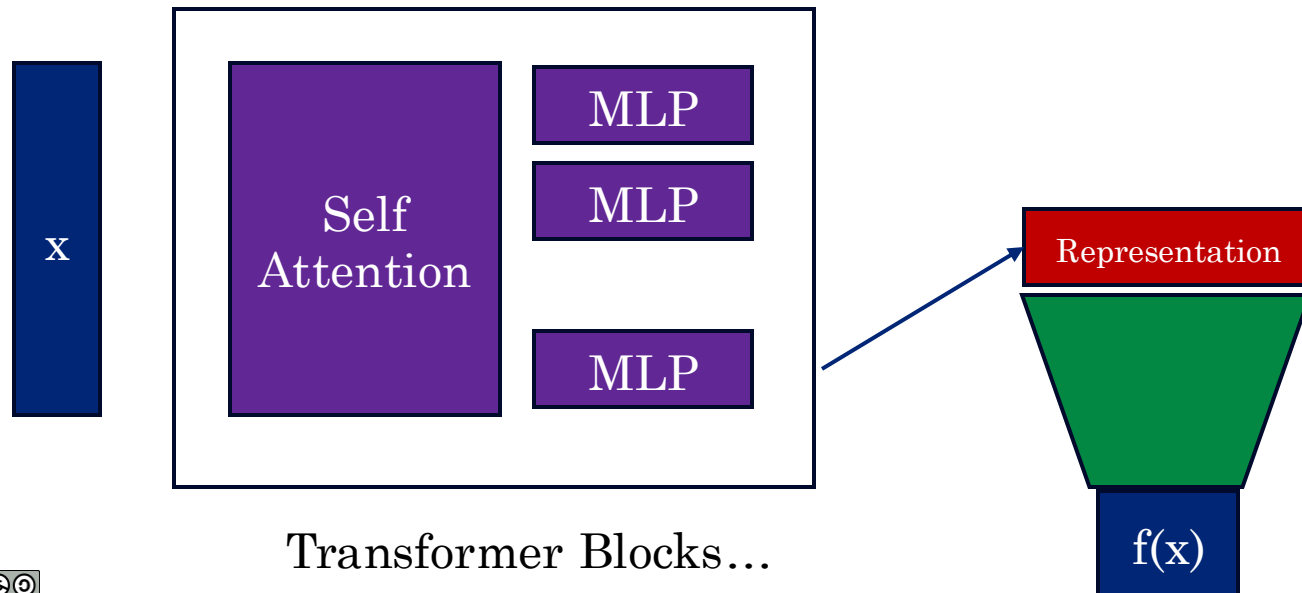
The key idea of self-supervision

In supervised learning you start with $\{\mathbf{x}, y\}$ pairs.

Given features $\mathbf{x}$, predict a label y .

Train a model so that $f(\mathbf{x})$ is close y .

Min sum $||f(\mathbf{x}_i) - y_i||^2$ or more generally Min Trainloss $(f(\mathbf{x}), y)$ using gradient descent.



Self-supervision:

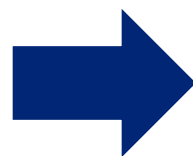
To get a good representation we create a fake supervised task.

Predicting the Next Word is *Knowledge*

The capital of California is Sacramento
San Francisco (1862)
Benicia (1853)
Vallejo (1852)

Predicting the next word allows you to:

- Answer questions
- Tell stories
- Accomplish tasks



Generative AI

token

How do we *model* the next ~~word~~?

Quick Recap

- **Causal language modeling** predict next token given context

Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})]$

- **Auto-regressive** (iterative) decoding:

The best class at UC Berkeley **is**

The best class at UC Berkeley is **EECS-189**

The best class at UC Berkeley is EECS-189!

The best class at UC Berkeley is EECS-189!<**stop**>

Re-explaining

Predicting the Next Token with a Transformer

Modeling the next token

Model: $\Pr(\underbrace{\text{"hat"}}_{\text{Next Token}} \mid \underbrace{\text{"the cat in the"}}_{\text{Context (ordered tokens)}})$

Step 1: Tokenization of the Context

Modeling the next token

Model: $\text{Pr}(\underbrace{\text{"hat"}}_{\text{Next Token}} \mid \underbrace{\text{"the cat in the"}}_{\text{Context (ordered tokens)}})$

Step 1: Tokenization of the Context

	the	cat	in	the
	↓	↓	↓	↓
Token Id:	142	307	153	142

Embedding Table Lookup

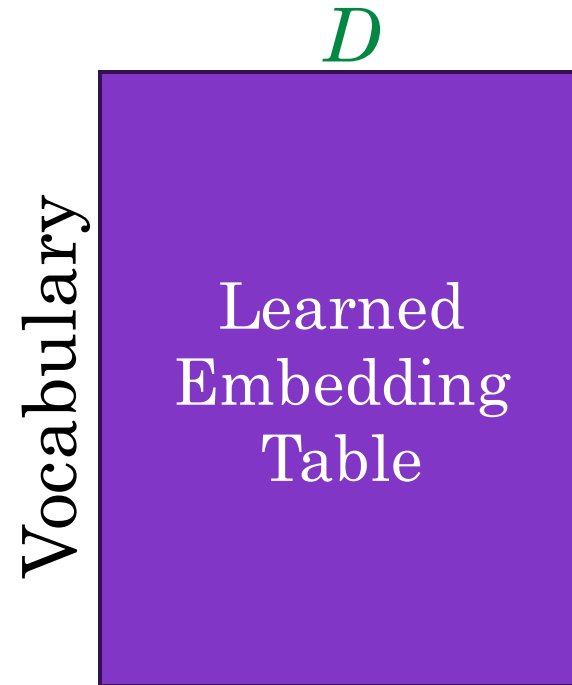
Lookup **embedding** is **learned embedding table**.

the → 142

cat → 307

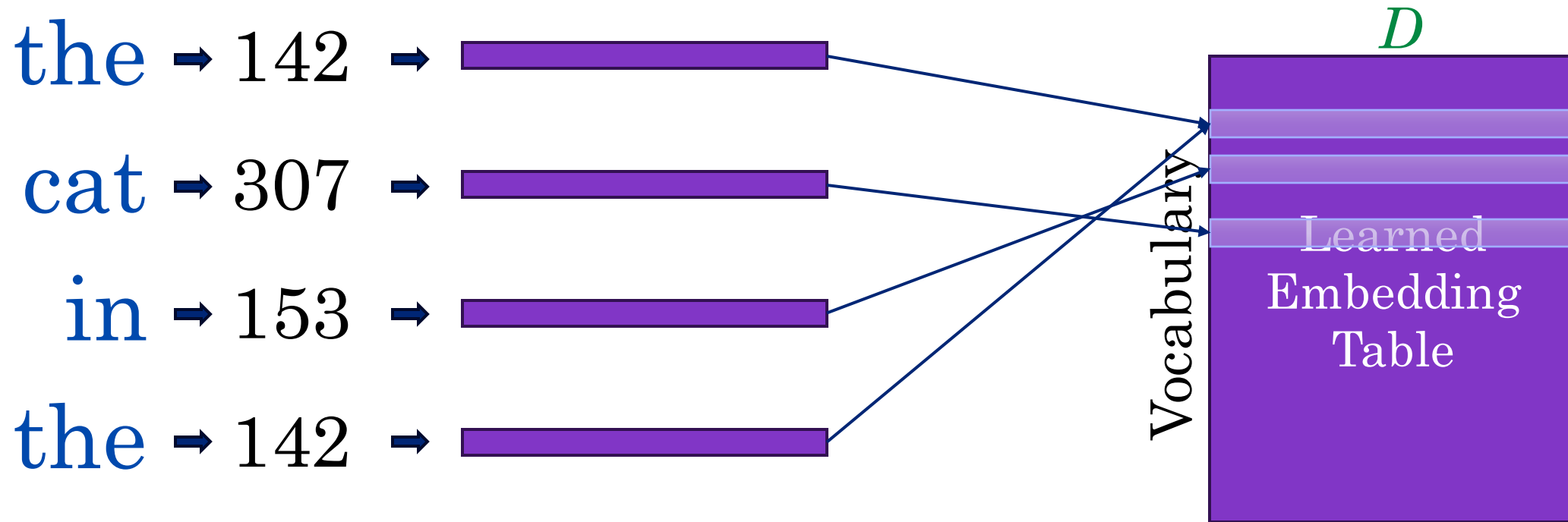
in → 153

the → 142



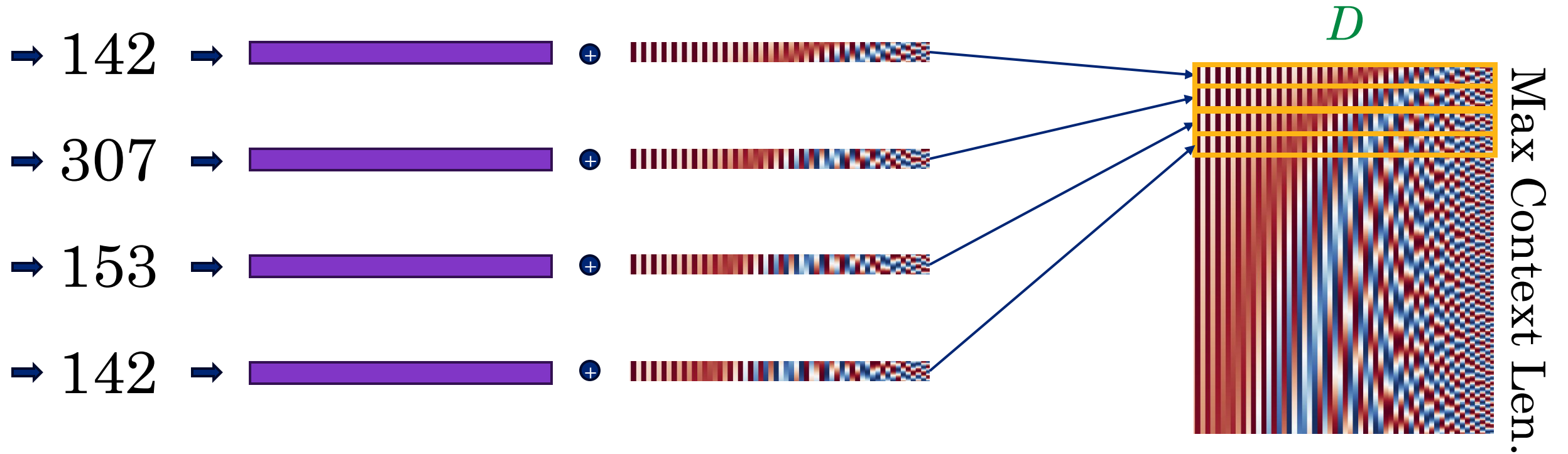
Embedding Table Lookup

Lookup **embedding** is learned embedding table.



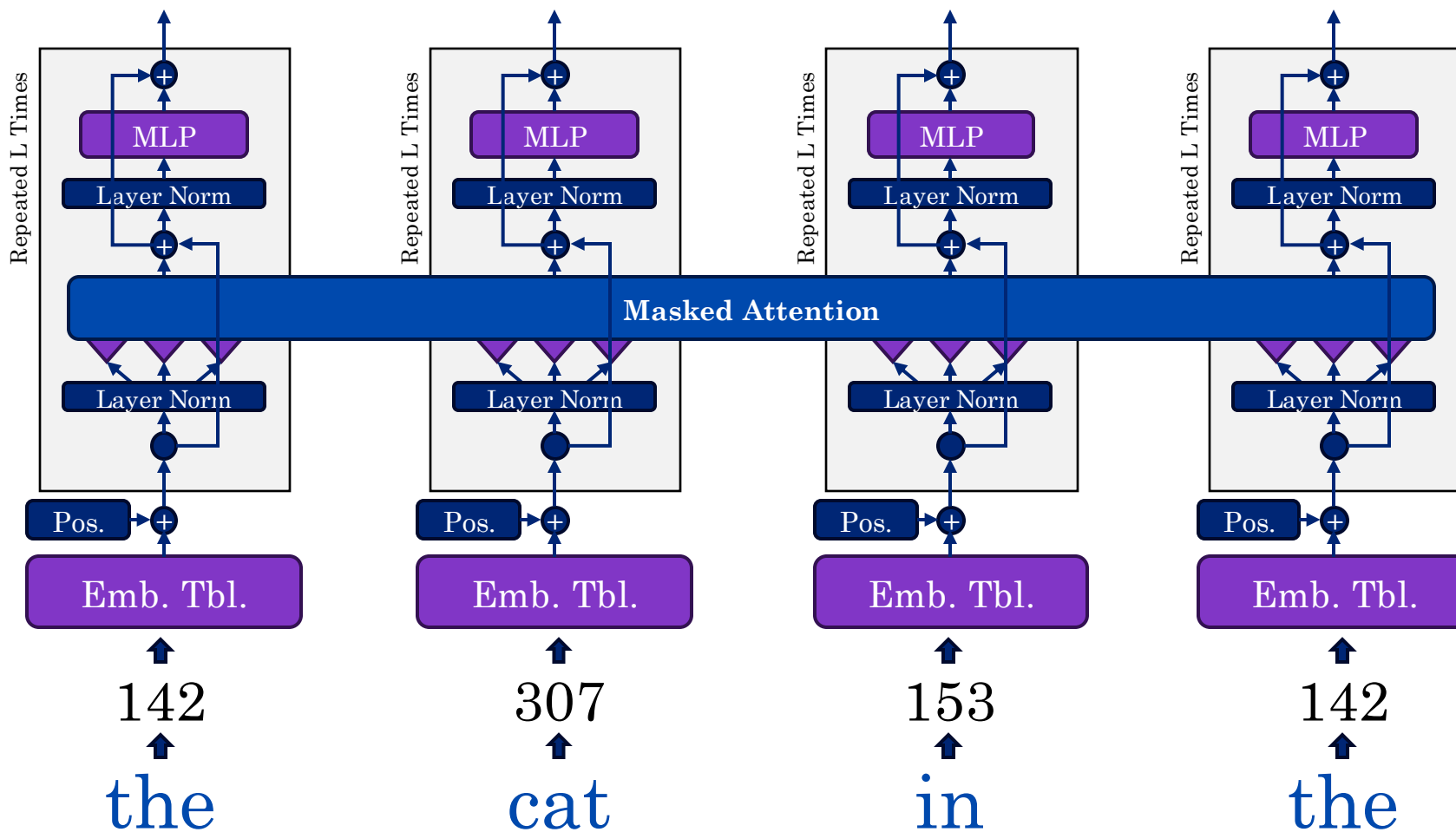
Positional Encoding (optional)

Add fixed positional encoding to embeddings based on position

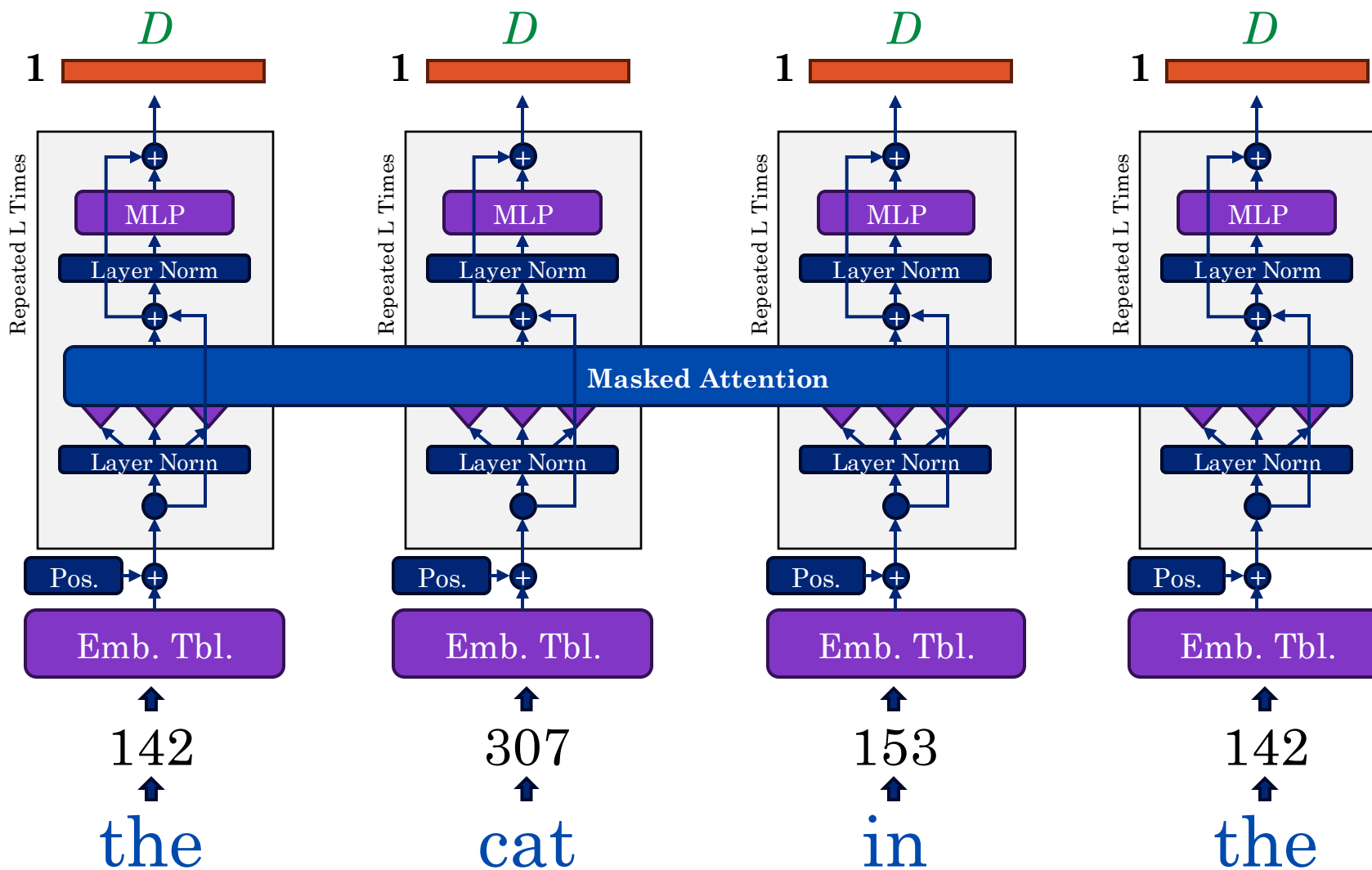


Pass into transformer architecture

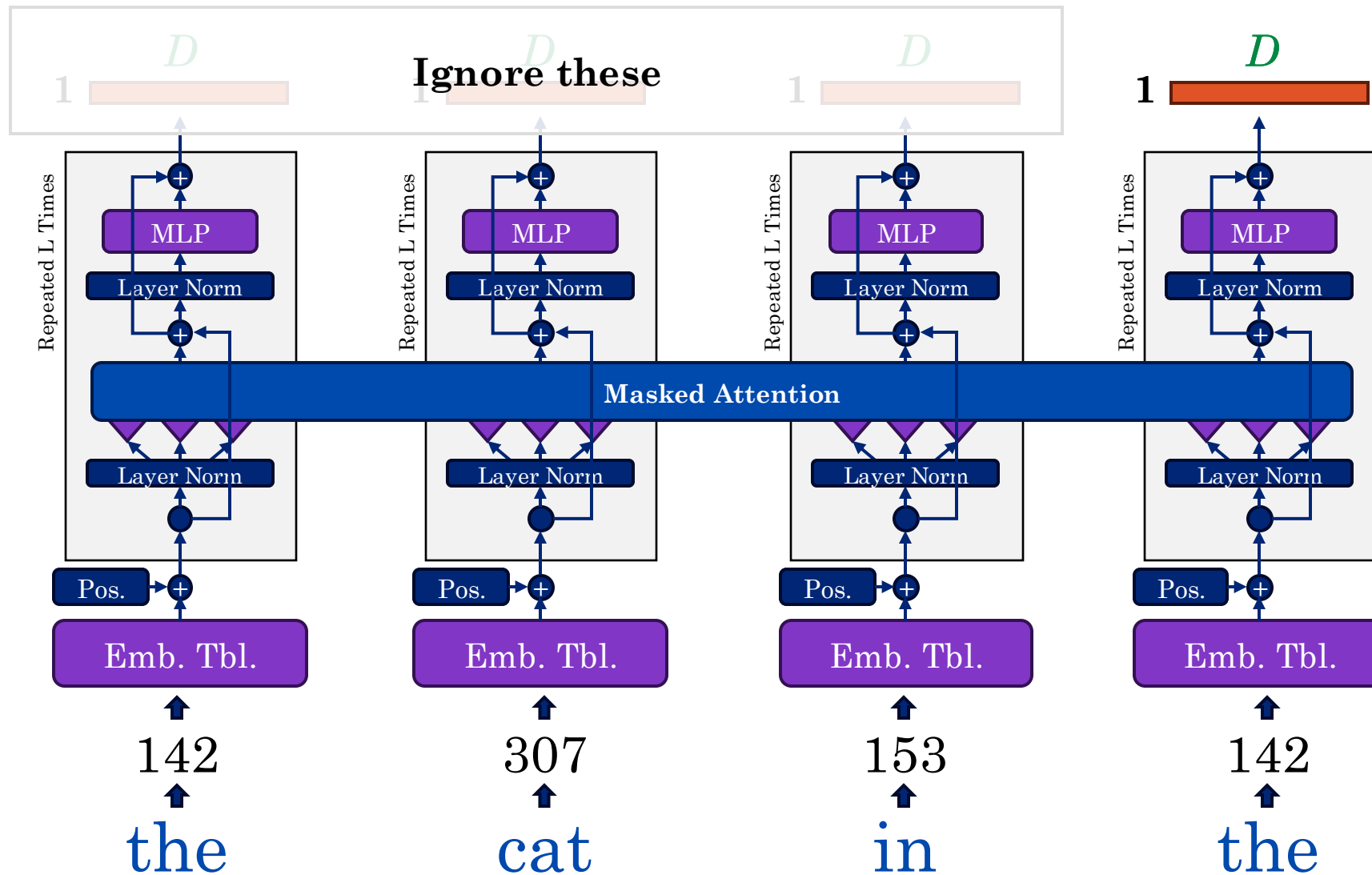
Predicting the Next Token (Transformer)



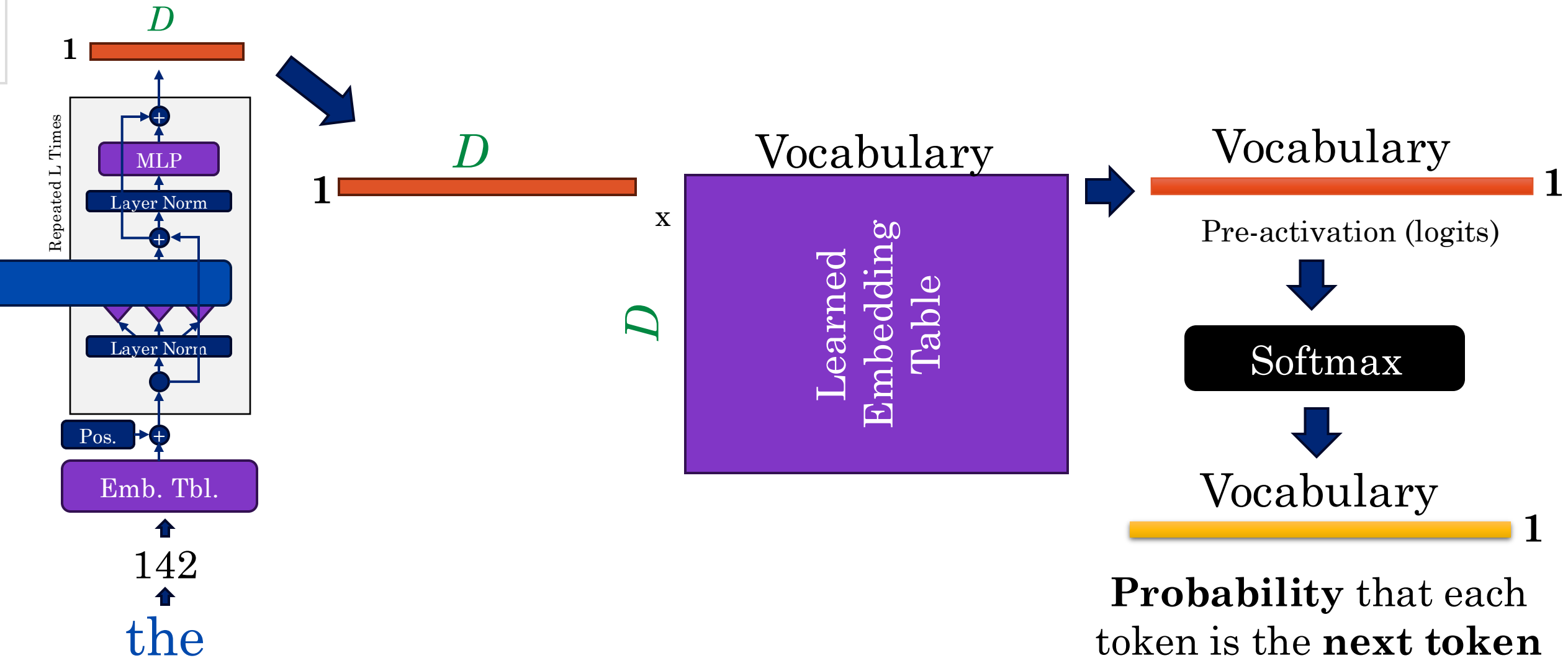
Predicting the Next Token (Transformer)



Predicting the Next Token (Transformer)



Predicting the Next Token (Transformer)



Training the Model

What is Chat GPT?

Chat: natural language system

✓ **G: Generatively** – Designed to model the creation of text

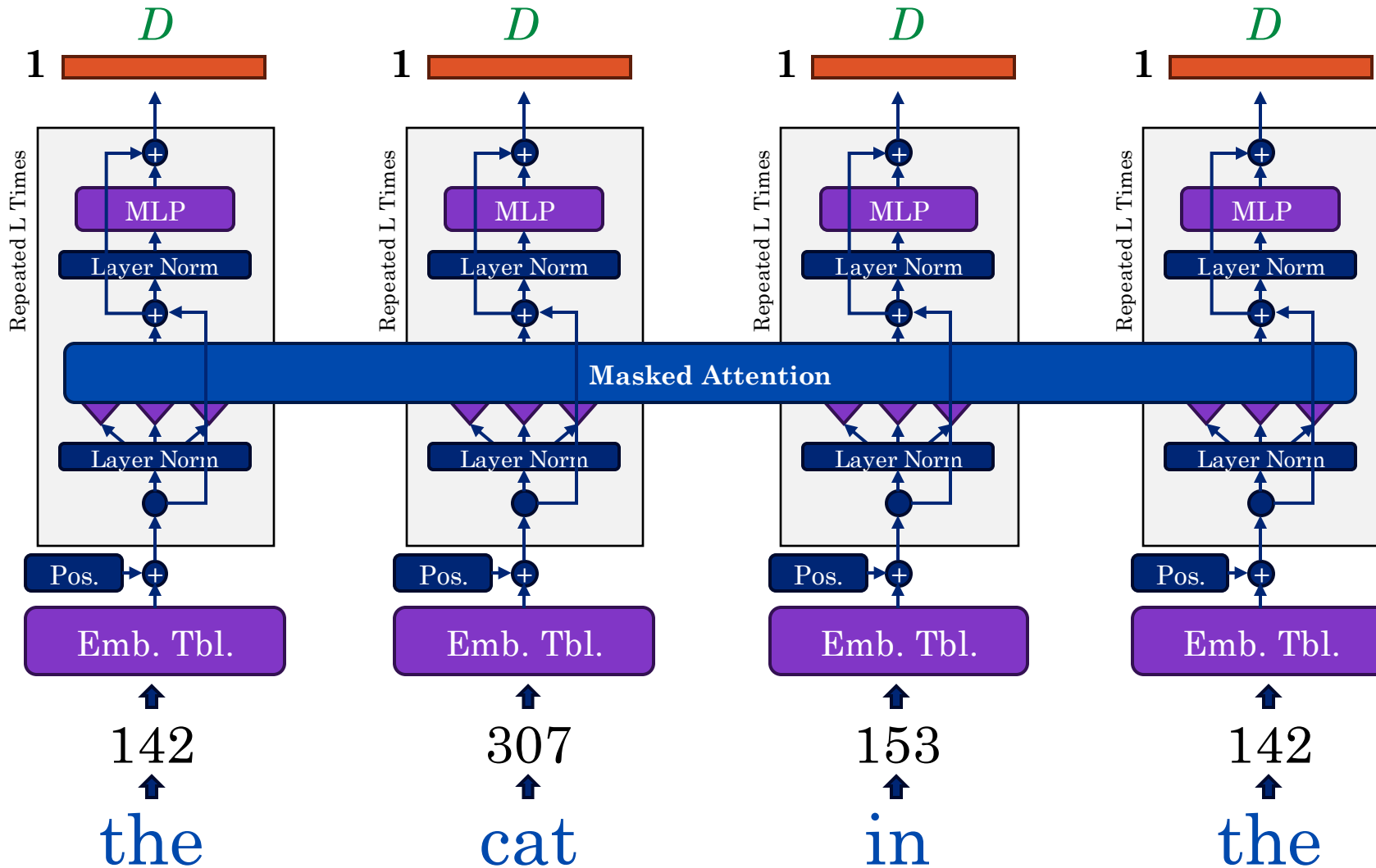
P: Pretrained – Trained on lots of naturally occurring data

✓ **T: Transformer** – A kind of neural network architecture

Chat GPT is just one example of a

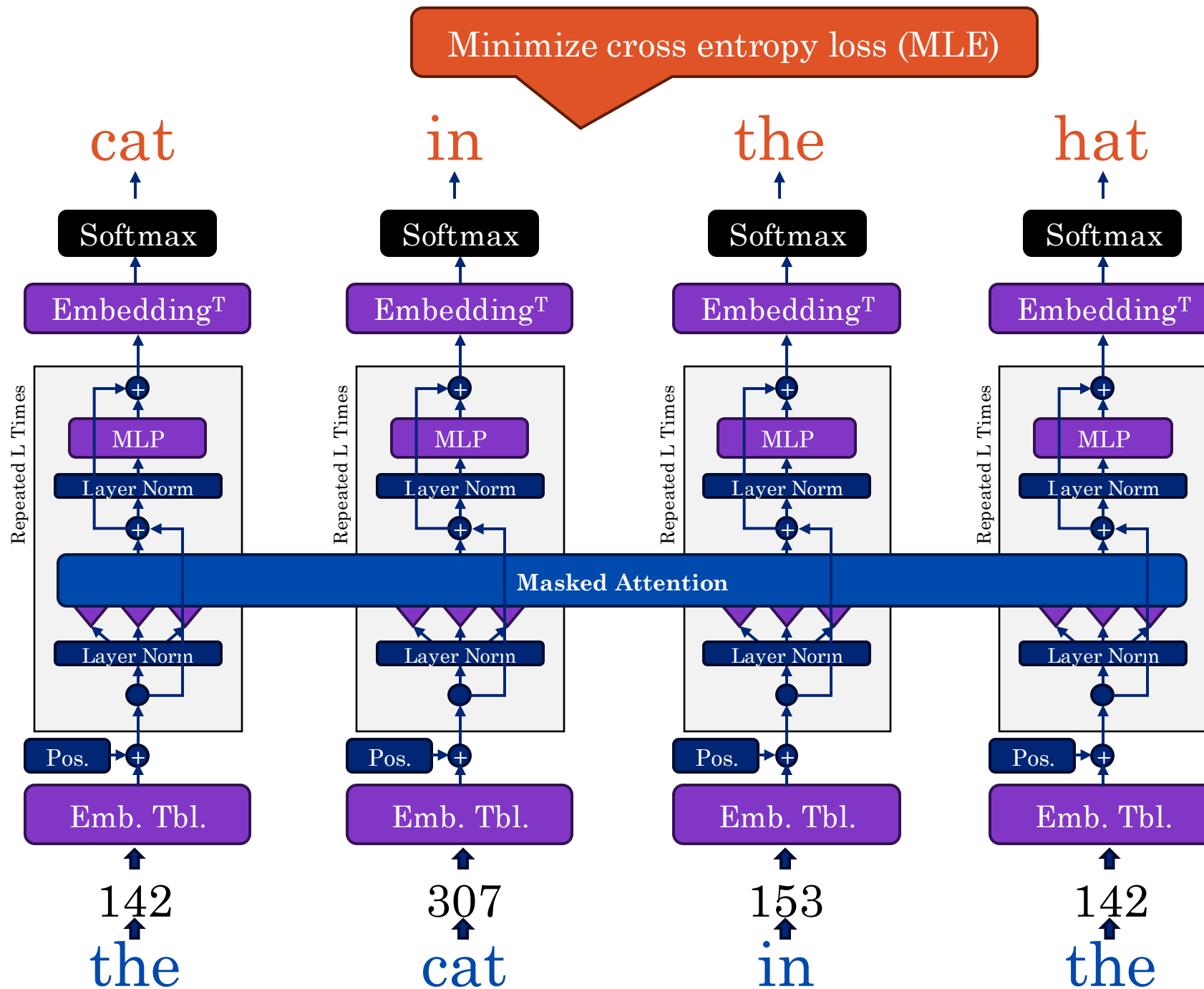
✓ **Large Language Model (LLM)**

Training the Model

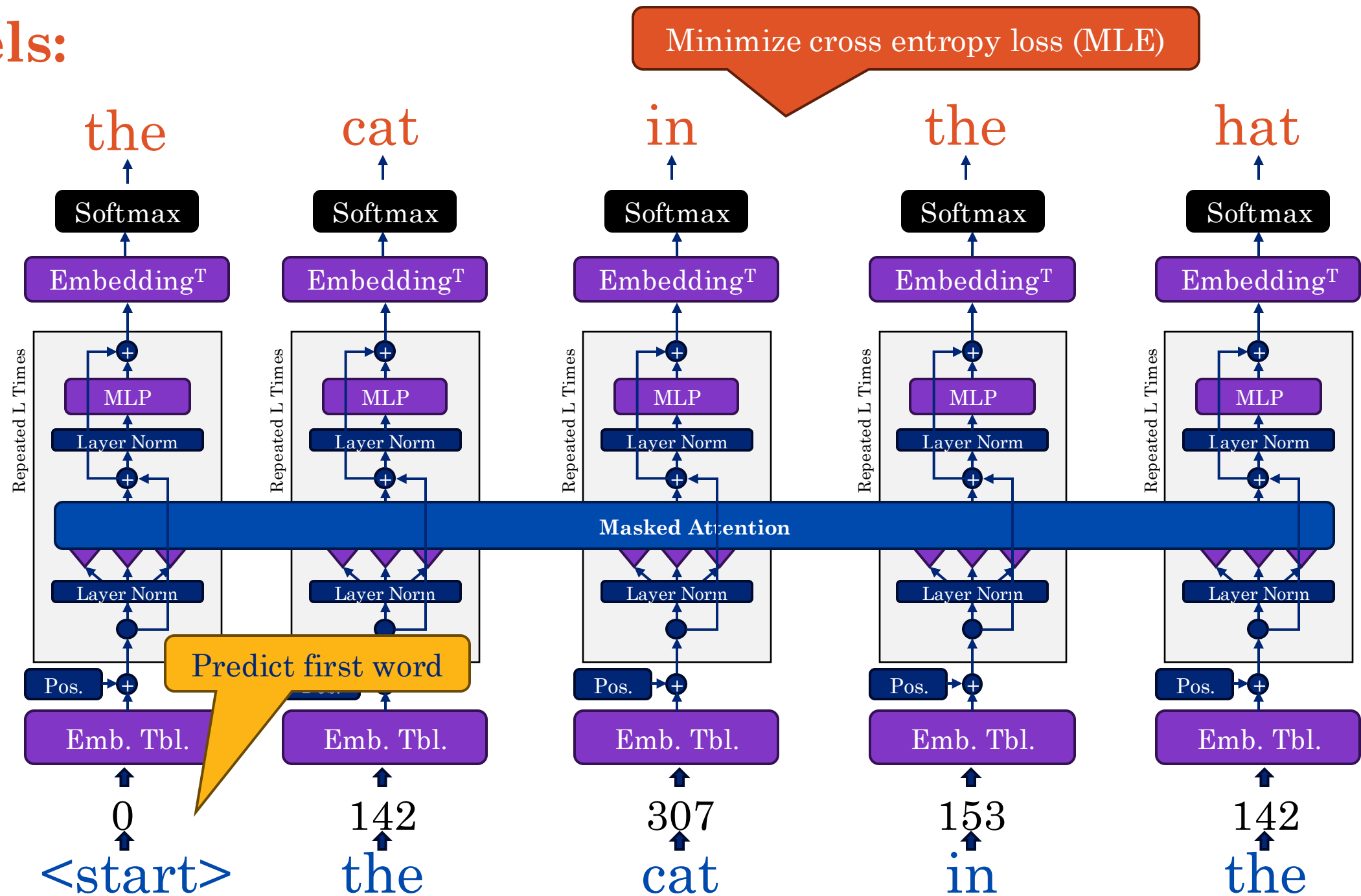


Training the Model

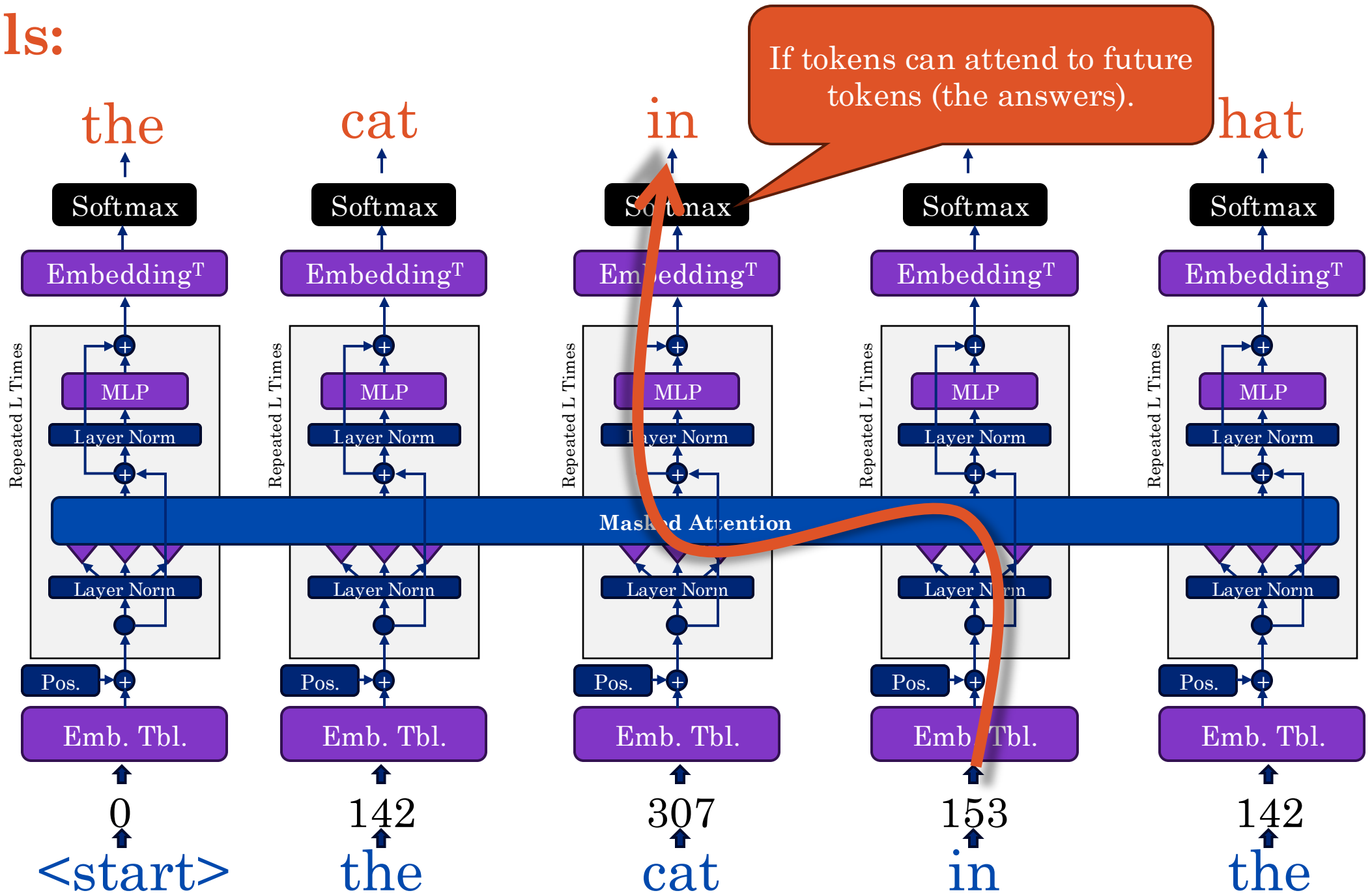
Labels:



Labels:



Labels:



The Masked Attention

$$\text{Attn}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = (\boldsymbol{\alpha} \mathbf{V})_n = \sum_{i=1}^n \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^n \left(\text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

Symbols:

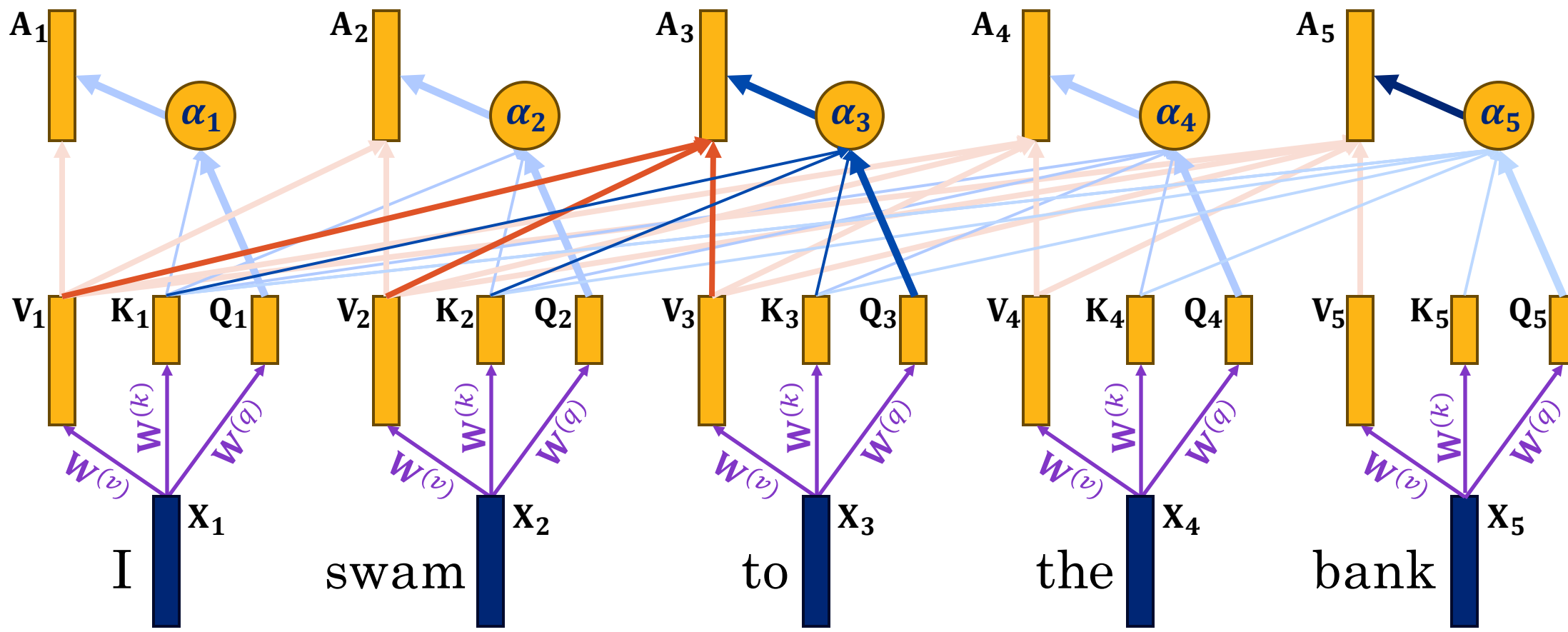
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

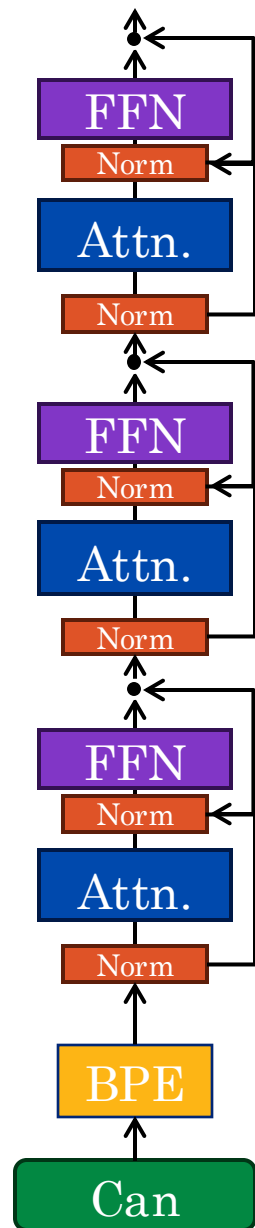
$$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}} \right]$$

$$\boldsymbol{\alpha} \in \mathbb{R}^{N \times N}$$

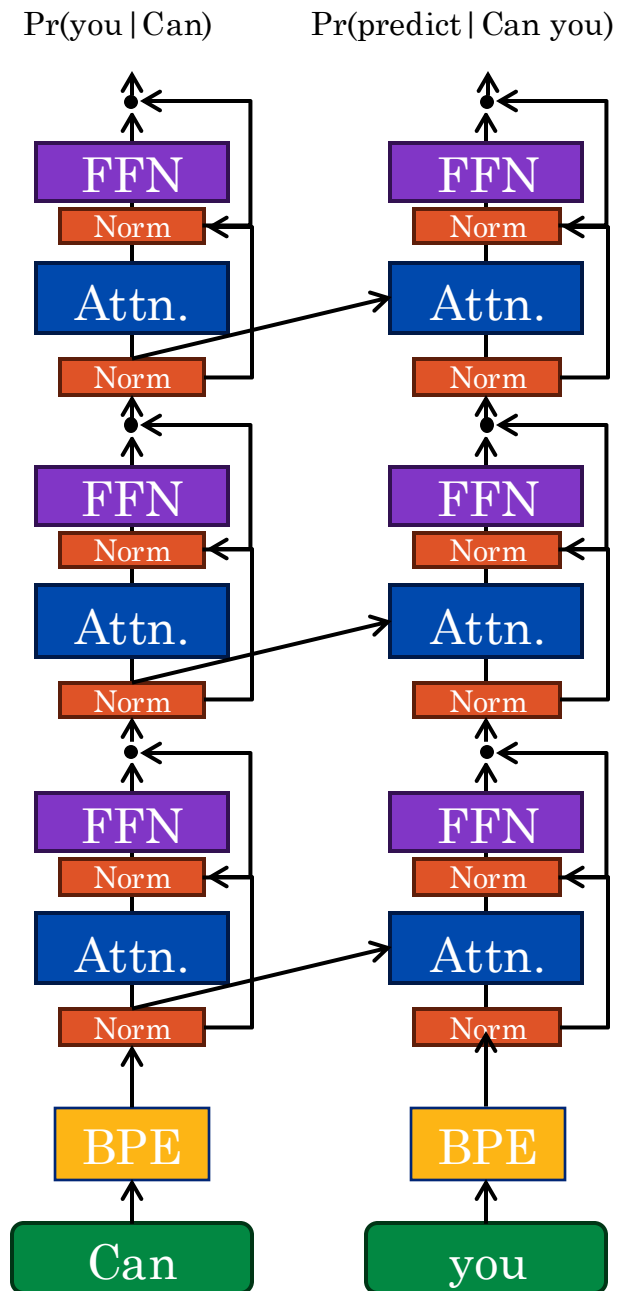


Unrolling the Model

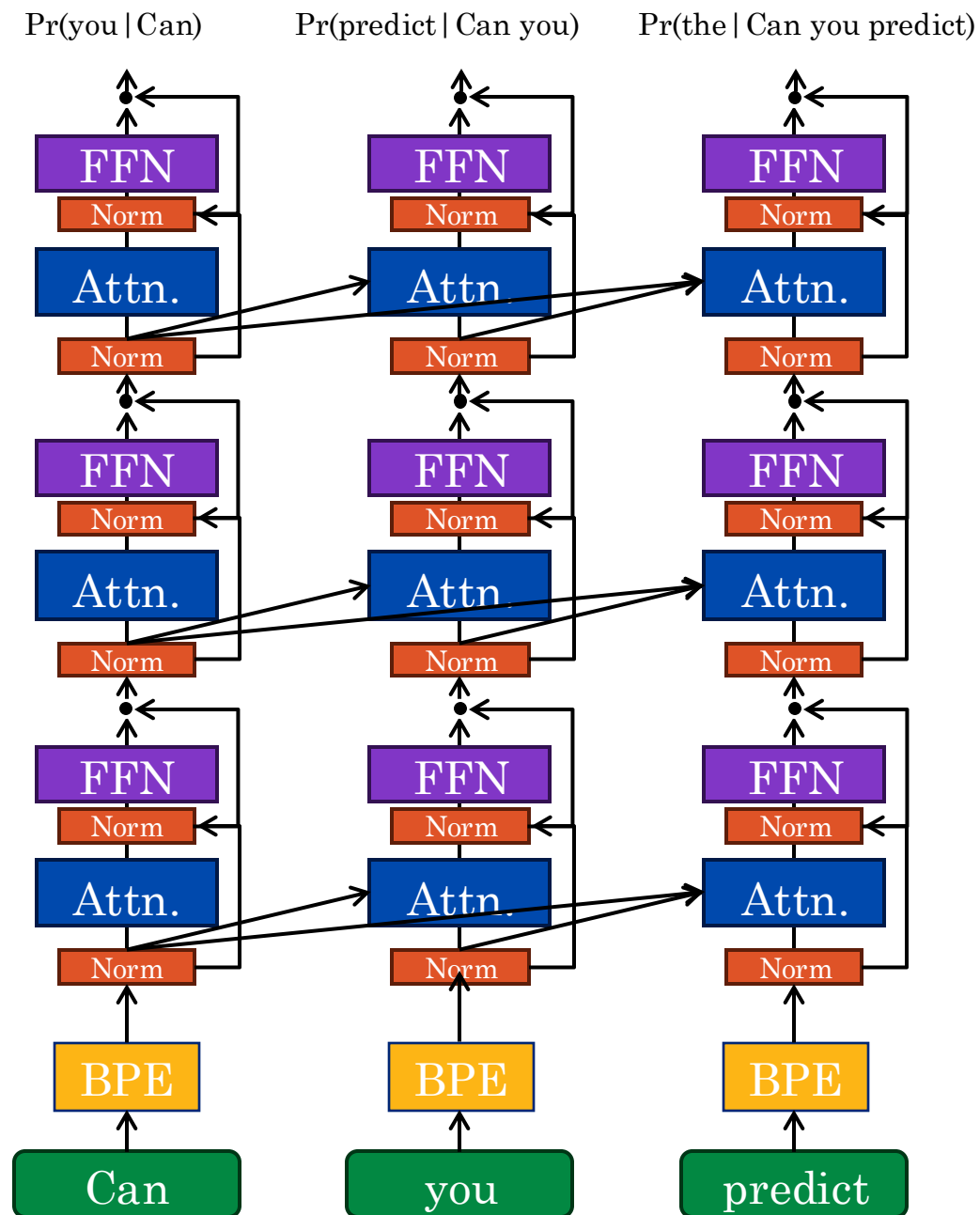
$\Pr(\text{you} \mid \text{Can})$



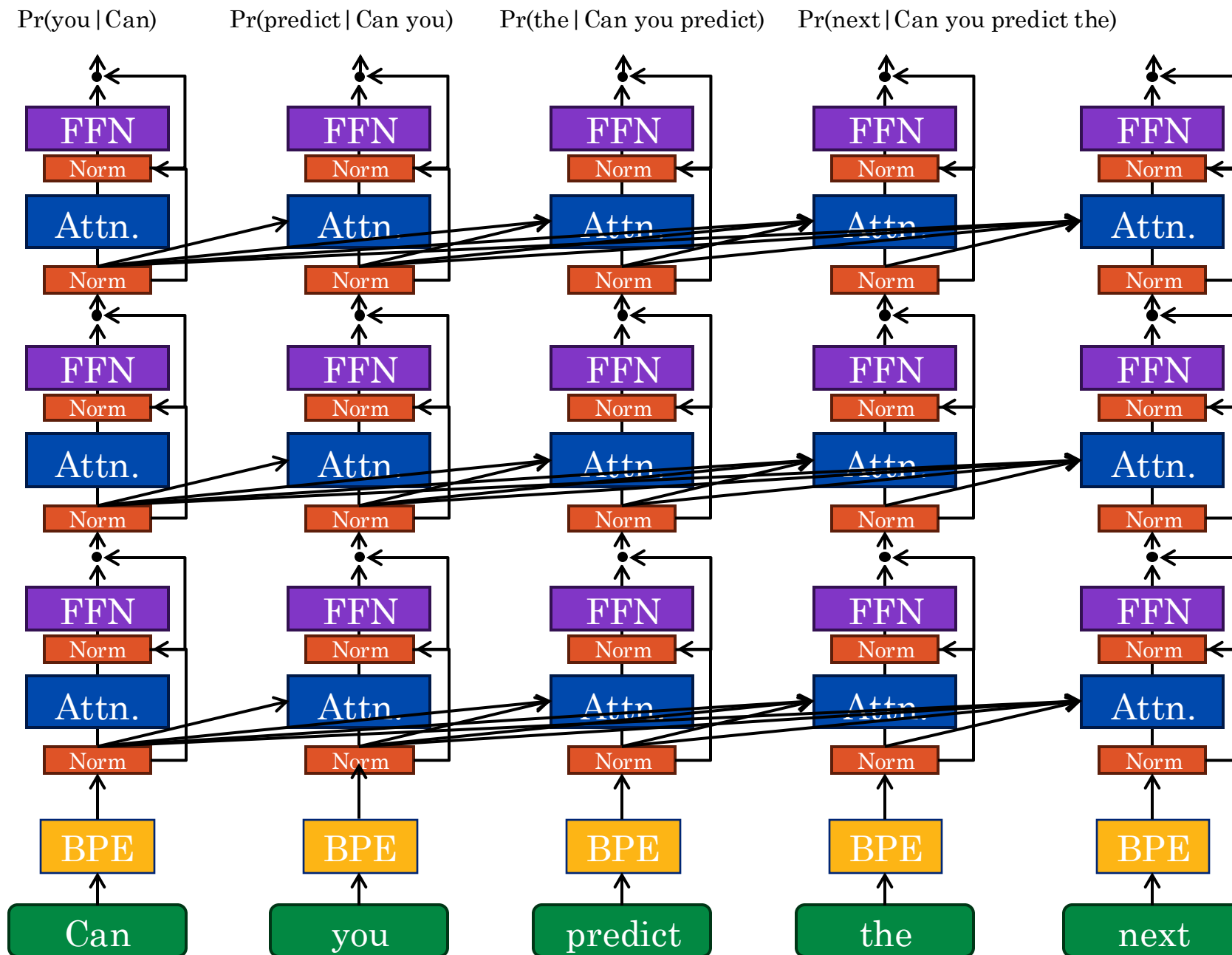
Unrolling the Model



Unrolling the Model

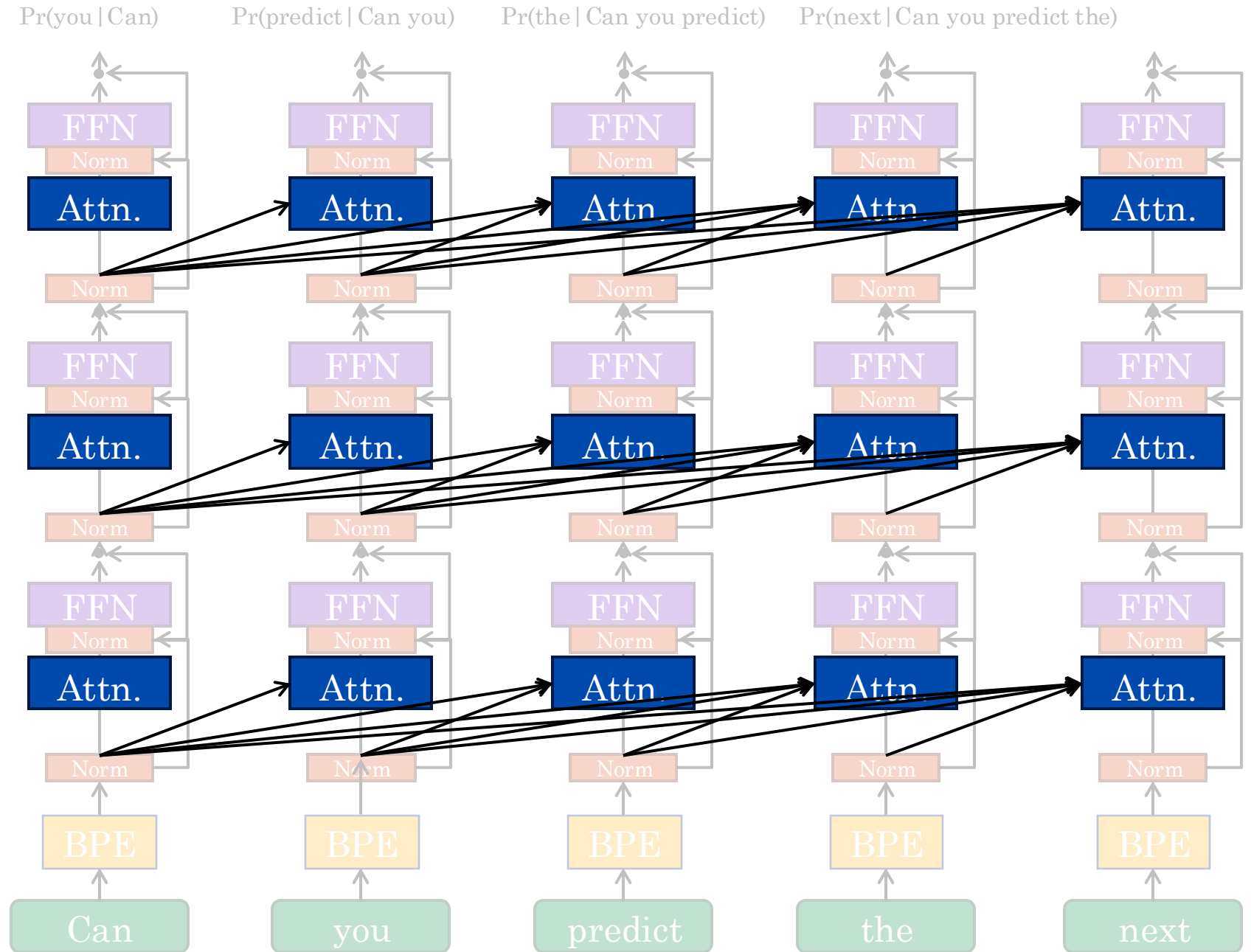


Unrolling the Model

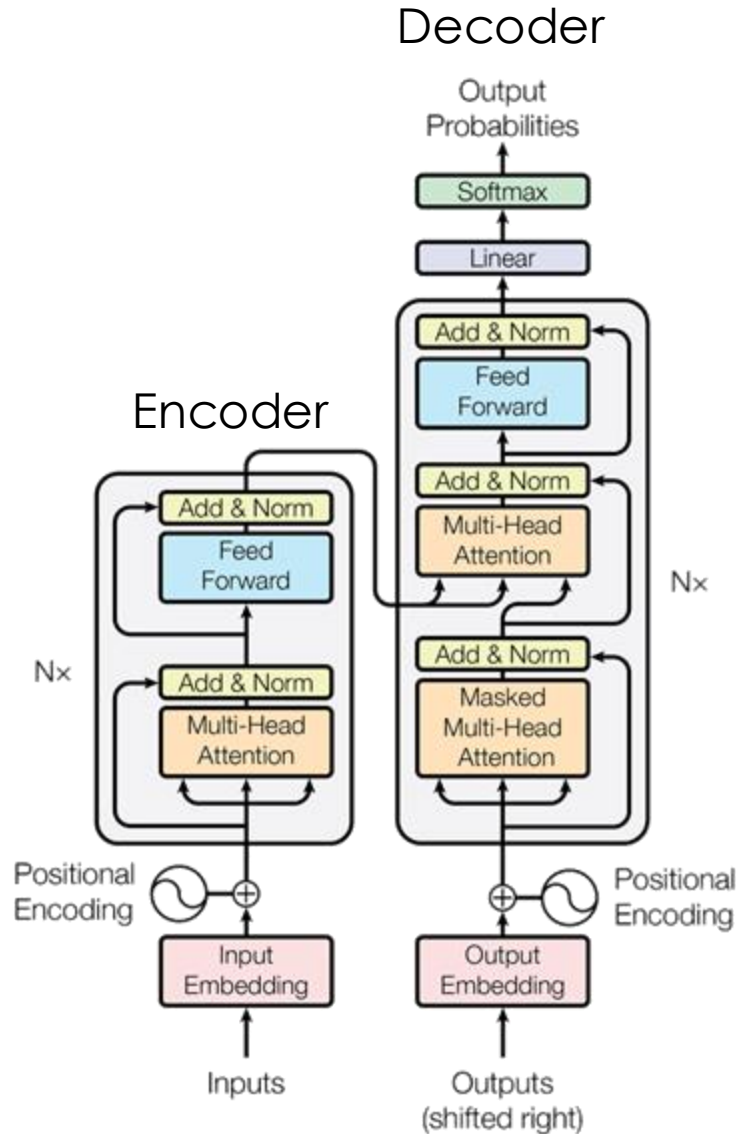


Masked Attention

Auto-regressive
Attend only to
previous tokens
Decoder only
Transformer

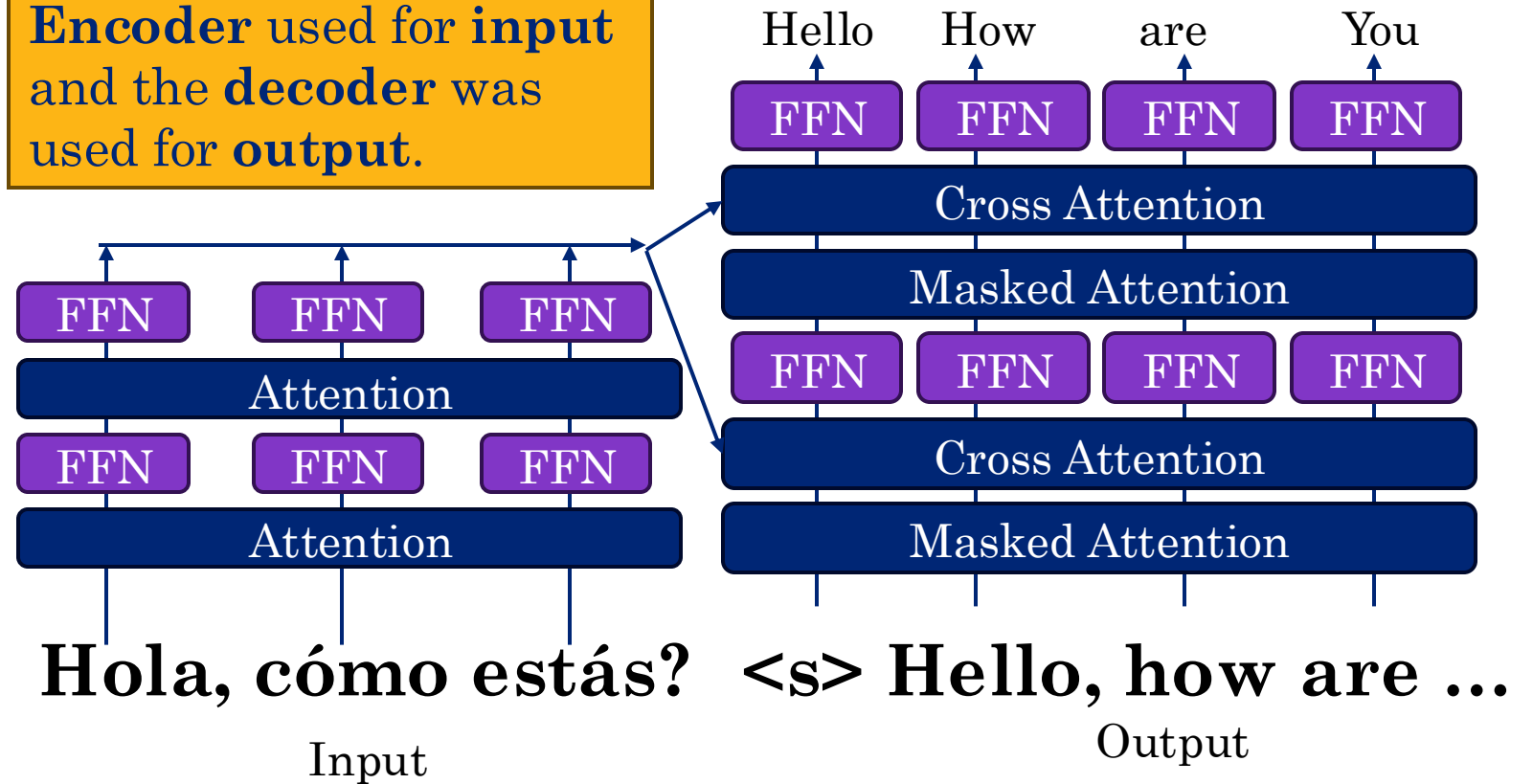


The Encoder-Decoder Transformer



The original transformer paper used both an **encoder** and a **decoder**. Why?

Encoder used for **input** and the **decoder** was used for **output**.



Visual Language Models

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.

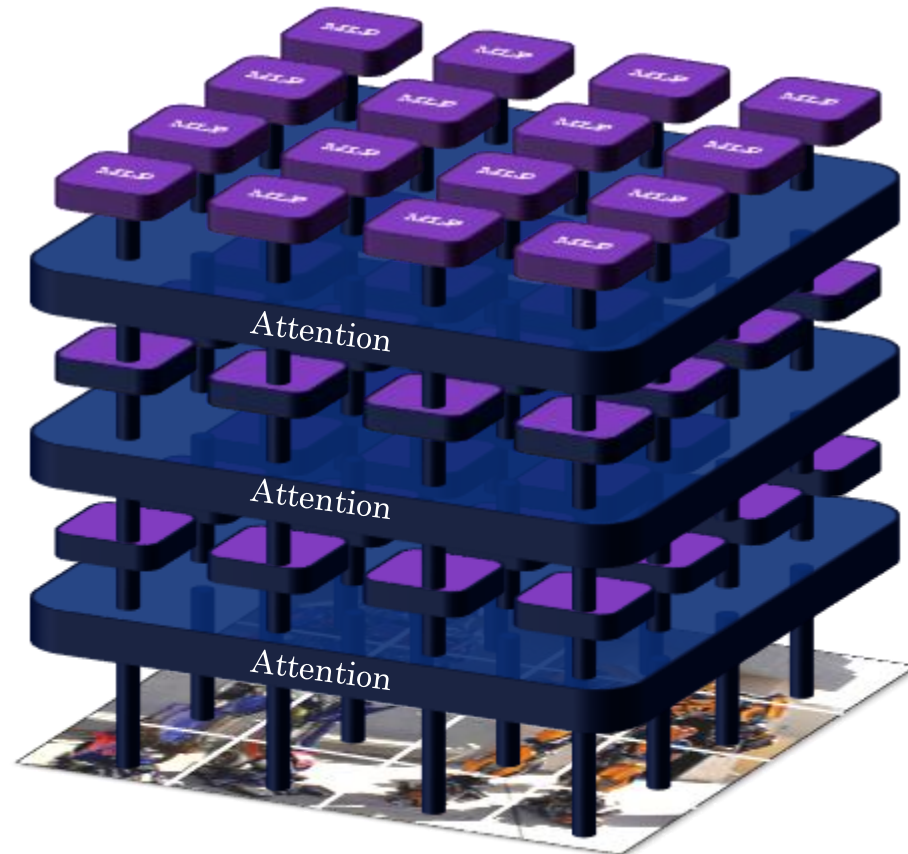
Describe this image:



Visual Language Models

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.

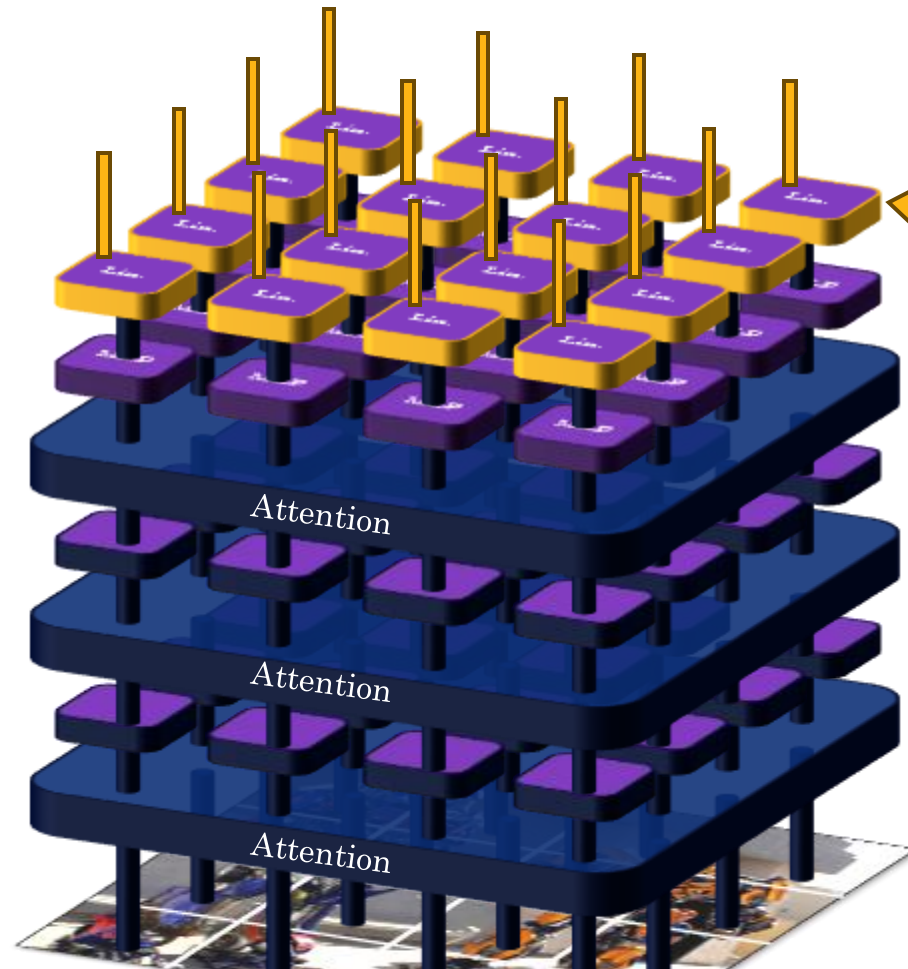
Describe this image:



Visual Language Models

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.

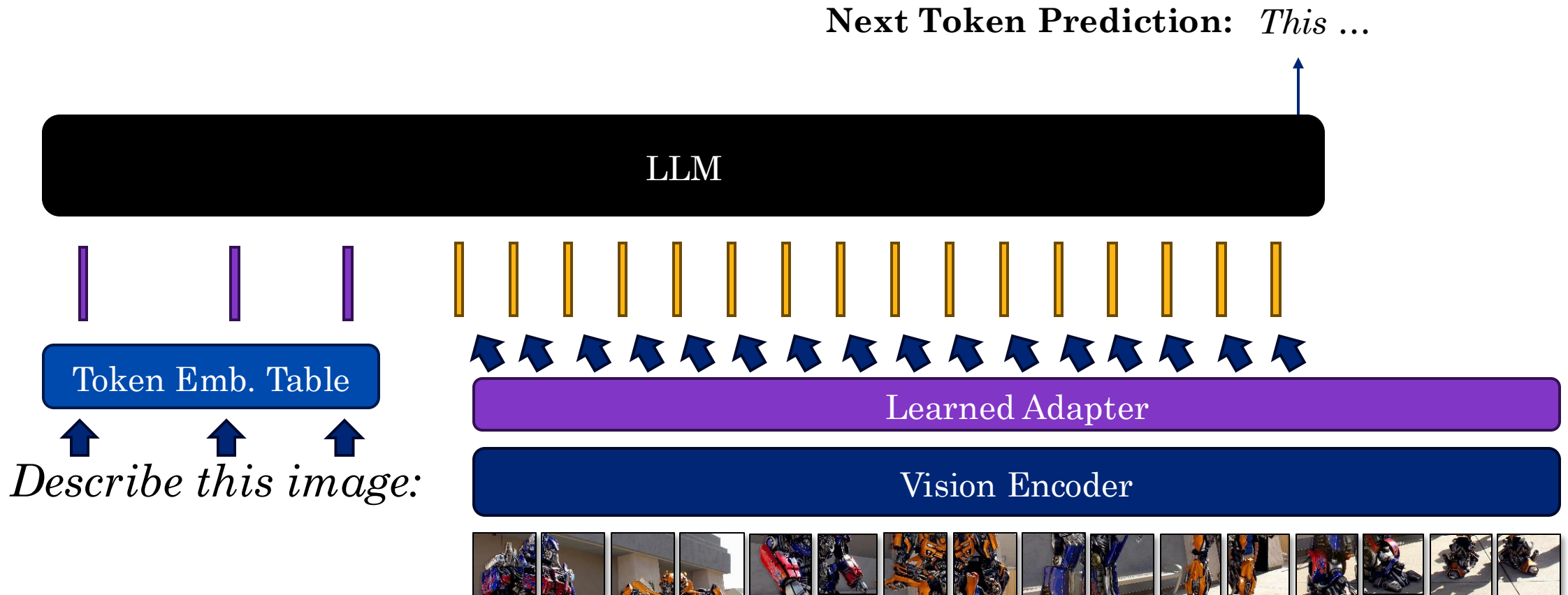
Describe this image:



Learn “adapter” to transform image embeddings to token embeddings.

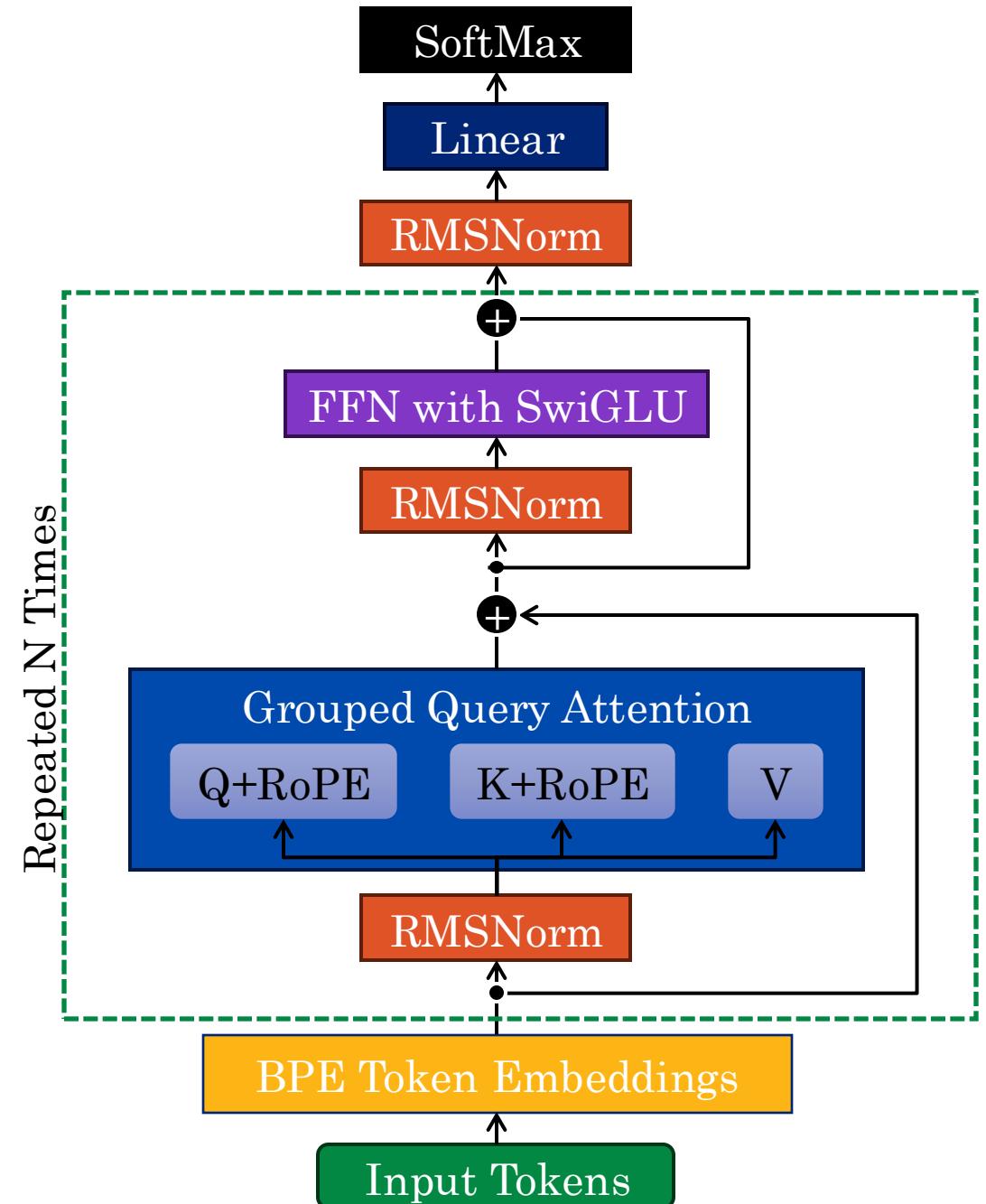
Visual Language Models

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.

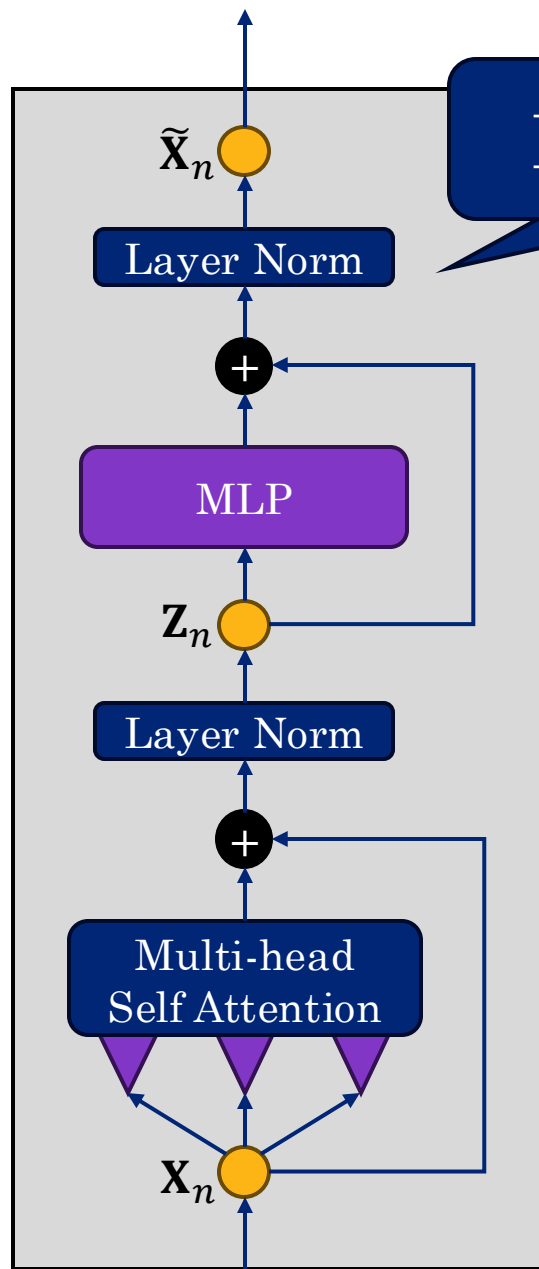


Llama-3 Architecture

- **RMSNorm** (Layer Norm.) – improve training stability
- **FFN with SwiGLU** – Feed forward network
- **Residual Connections** – improve training stability
- **Repeated N Times** – increase model size

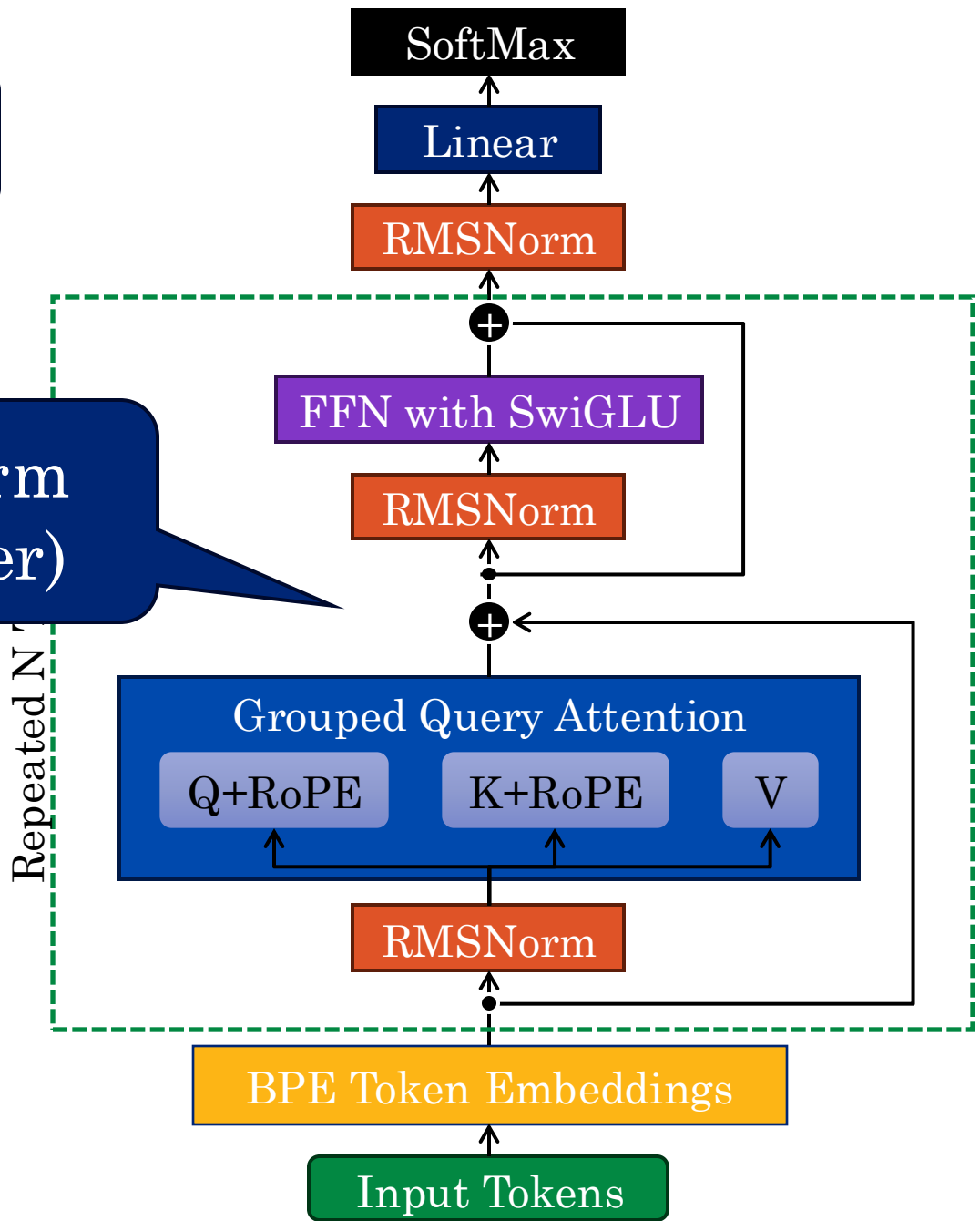


See [Actual Code](#) (its just one python script!)



Post-Norm

Pre-Norm
(is better)



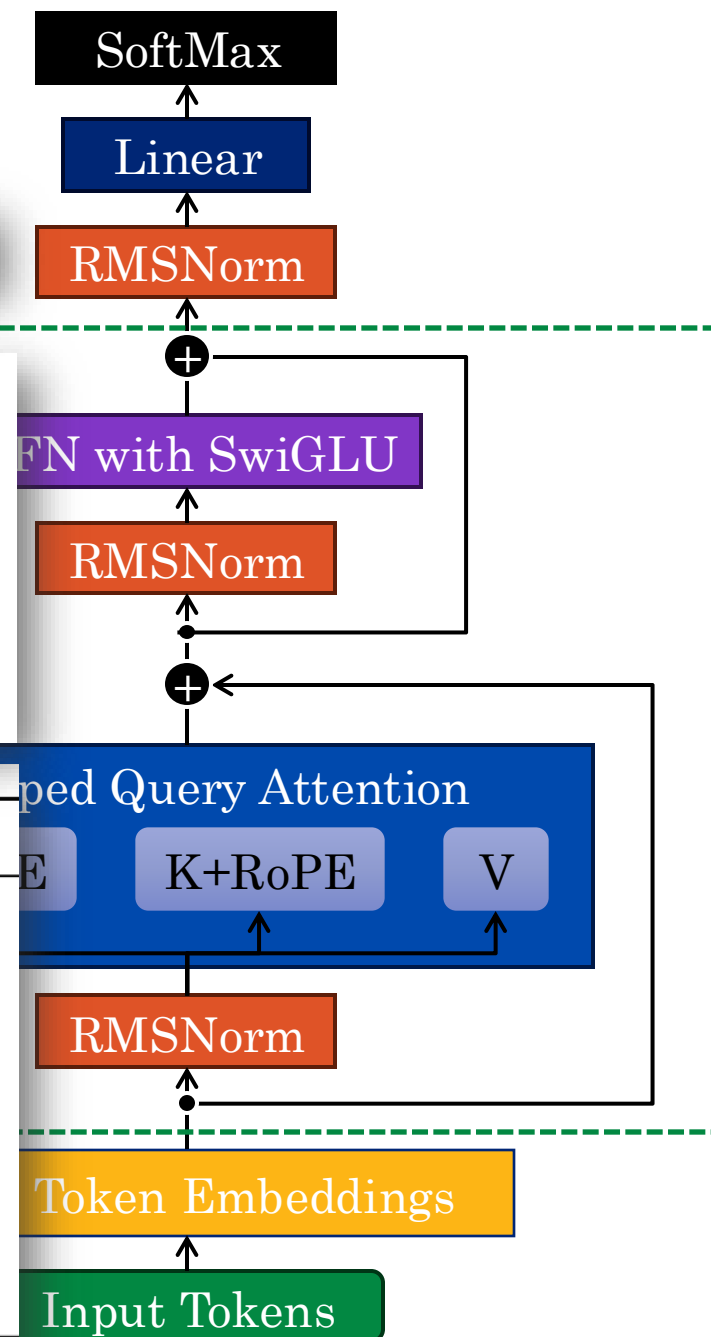
Going Big!

Llama-3 70b Instruct: 8192 hidden size, 80 layers, 64 query heads, 8 kv heads.

Llama 2

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128



Pre-training on *Everything**

Train the model on a **large collection** of data to learn **generalizable patterns**

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Reddit
Links

OpenAI
GPT3
Data Mix

Llama-3 “open-source” models trained on **15.6T tokens** from **an unknown data mix**.

- [405-billion parameter model](#) trained on 16K H100s (\$25K each)
- **39.3M GPU hours**

*Everything that is legal to use for training hopefully...

Now you know

What is Chat GPT

There is still one more thing

Chat: natural language system

- ✓ **G: Generatively** – Designed to model the creation of text
- ✓ **P: Pretrained** – Trained on lots of naturally occurring data
- ✓ **T: Transformer** – A kind of neural network architecture

GPT alone can't chat!

Post-training

Tuning language models to be useful.

Need to Teach Model to **Follow Instructions (and be helpful!)**

- Generative pre-training **captures knowledge**
 - To finish the statement “Four score and seven years ago ...” you need to learn to memorize the text
- Resulting model predicts the rest of a statement.
 - “What is attorney client privilege?” the model might generate “Provide a concise answer using an example from class.”
- Use **Supervised Fine-Tuning** and **RLHF** to adjust “behavior” of model to **follow instructions** and **chat like a human**.

Supervised Fine-tuning (SFT)

Running **additional training iters.** with a specific task

The task differs from the original pre-training task

- **New objective:** translate sentence, follow instruction
- **New training data** from new source domain

Example: Chat conversations.

Smaller learning rate (as you get older you learn slower?)

- Avoid “catastrophic forgetting” – additional fine-tuning causes loss in pre-training knowledge and other post-training (e.g., instruction following) behaviors.

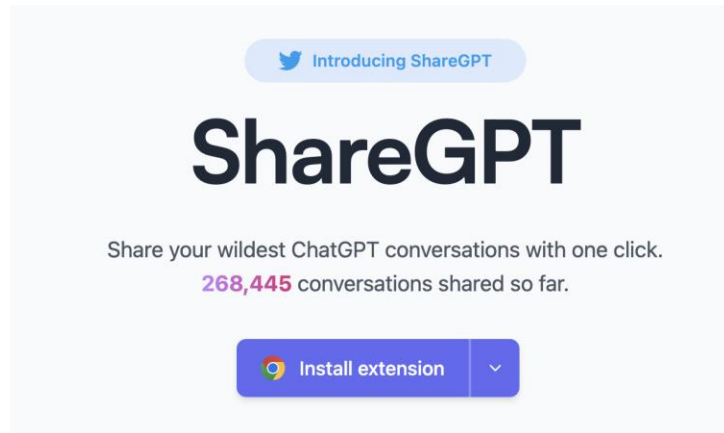


Vicuña

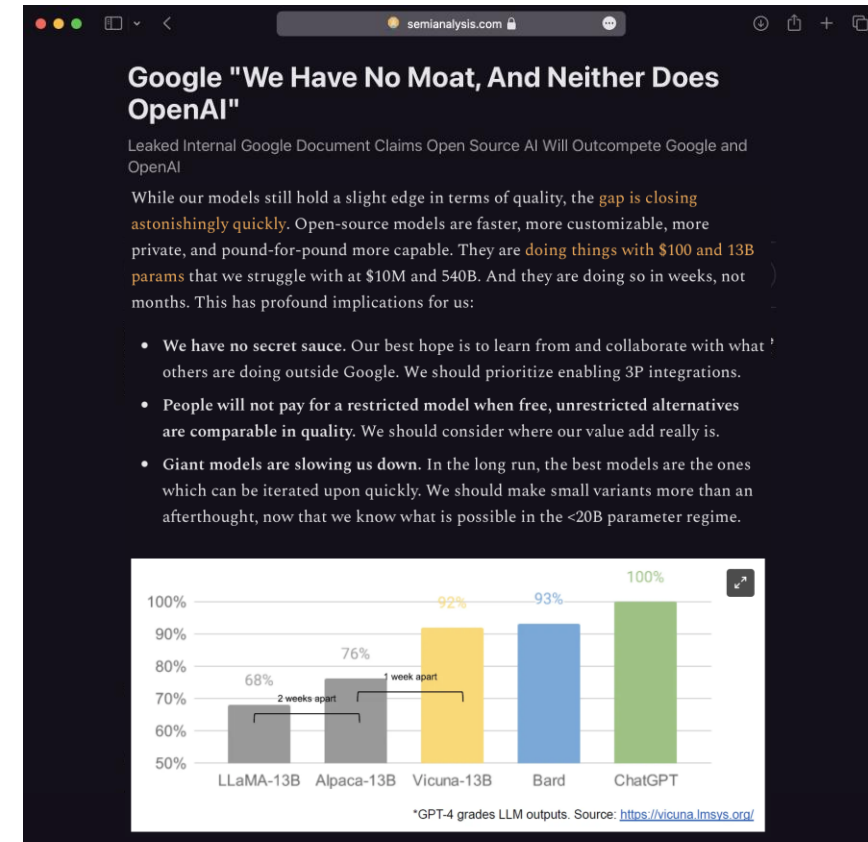
Open-source Instruction Fine-Tuned LLMs and LLM-as-a-judge

First open-source model that was
“comparable” to ChatGPT

Fine-tuned LLaMA-13B on ShareGPT Data



Tiny
High Quality Data
70K conversations
(~800MB)



Helped launch academic open-source GenAI research

LoRA Fine-Tuning

Reduce costs and catastrophic forgetting by fine-tuning a **low-rank perturbation**:

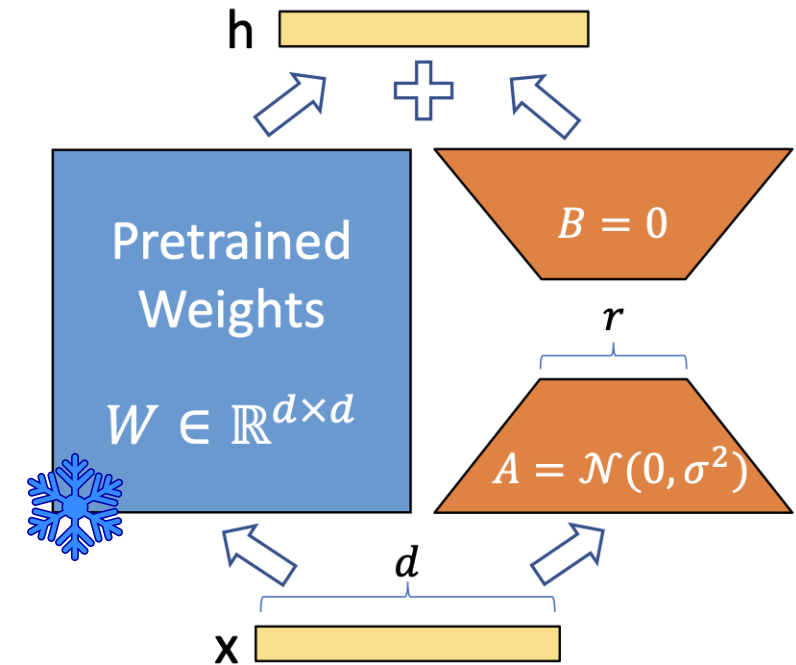
$$\mathbf{W}' = \mathbf{W} + \mathbf{A} \mathbf{B} x$$

where $\mathbf{W} \in \mathbb{R}^{D \times D}$ is the pre-trained weights and $\mathbf{A} \in \mathbb{R}^{D \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times D}$ are learned.

- The $\mathbf{B}$ weights are initialized to zero resulting in equivalent model (loss) prior to additional training.

Can be applied to **all weights** in the model.

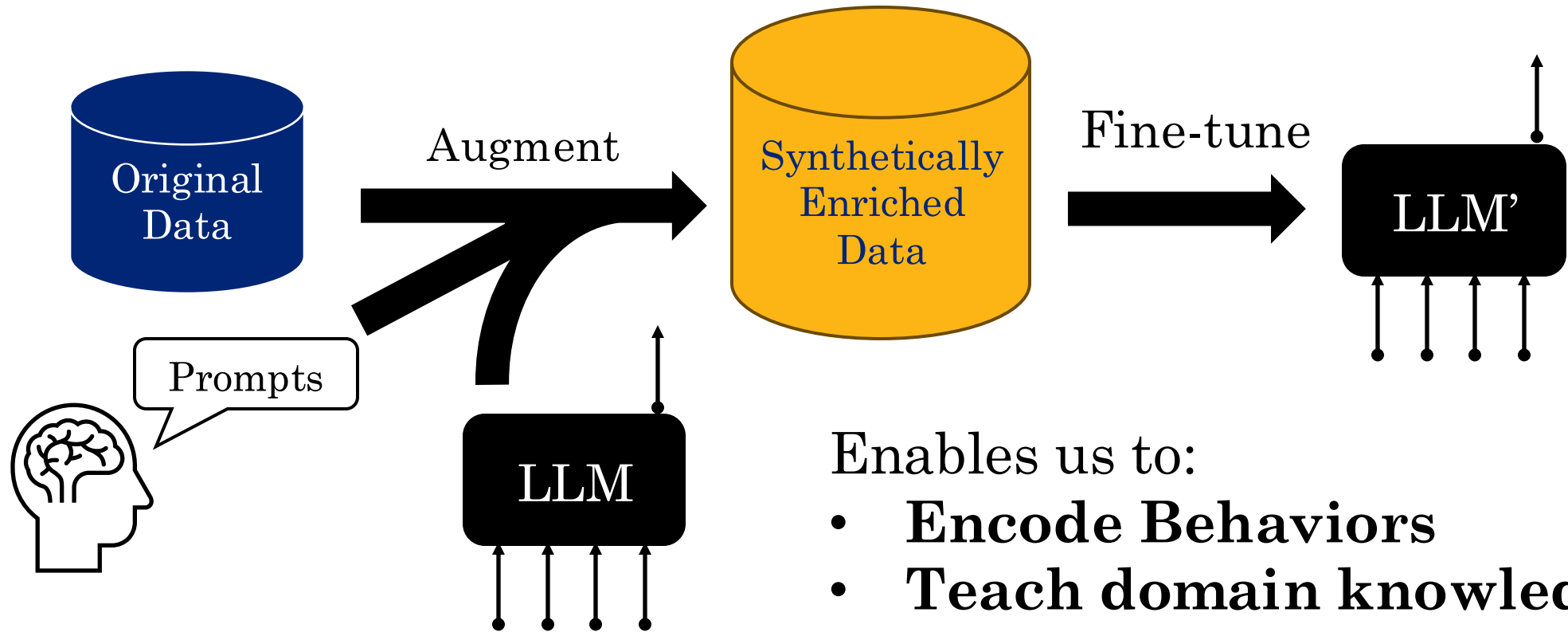
Reduces **training costs** and allows shared inference.



Synthetic Data Augmentation

(Programming GenAI with GenAI)

Using an LLM to **augment data** to **fine-tune** an LLM

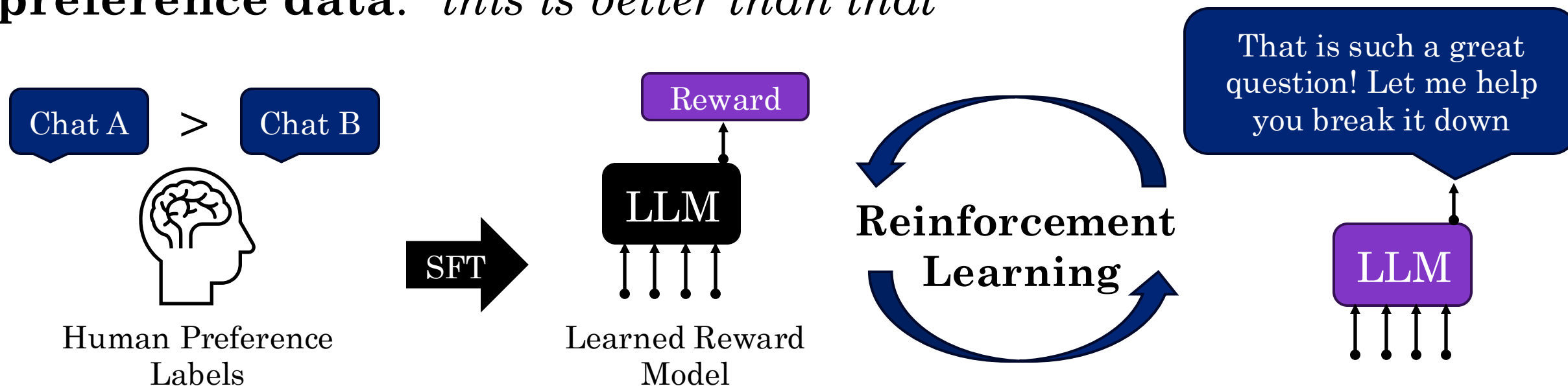


Enables us to:

- **Encode Behaviors**
- **Teach domain knowledge**

Reinforcement Learning from Human Feedback (RLHF)

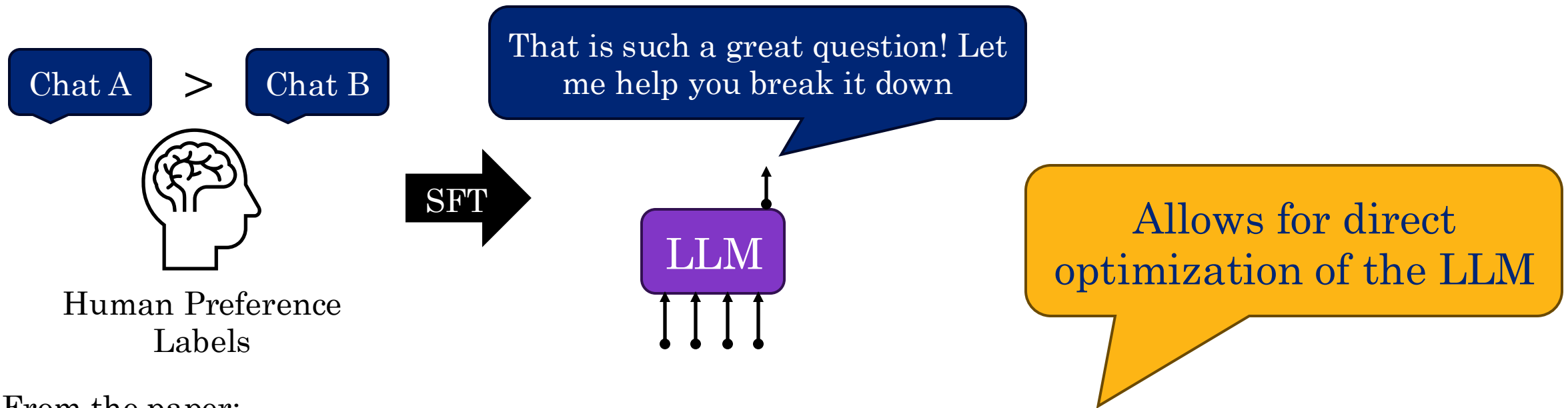
Can further **align model with human preferences** using **human preference data**: *“this is better than that”*



Helps to **make models more robust** and perform better in **safety situations** but can be **unstable** and **difficult** to train.

Direct Preference Optimization (DPO)

Directly apply **Bradley-Terry Model** (HW2) to recast preference learning as **Maximum Likelihood Estimation**.



From the paper:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Prompting and Test-time Compute

Using context to guide models and extract capabilities
through chain-of-thought

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

Zero-shot relies on model already
“knowing” how to complete the task.

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

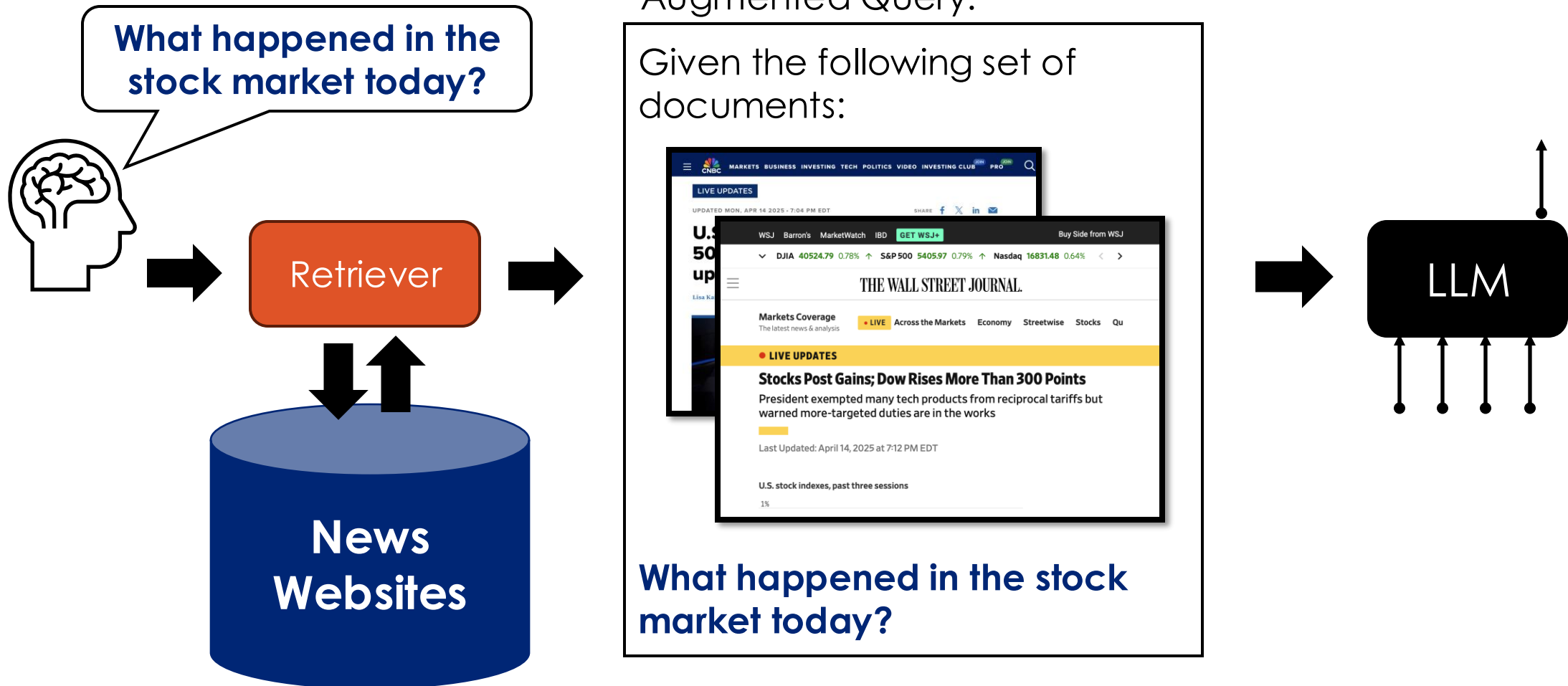
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

In-context Learning

Retrieval Augmented Generation (RAG)

Giving LLMs Specialized Knowledge



Chain-of-thought Prompting

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. 

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. 

Reasoning in Large Language Models

Popularized by ChatGPT-O1

Behavior: leveraging long chain-of-thought with planning, reflection, self-refinement, and back-tracking.

Prompt:

How many of the numbers from the set $\{1, 2, 3, \dots, 50\}$ have a perfect square factor other than one?

<|begin_of_thought|>

So I have this problem: "How many of the numbers from ...

First, I need to understand what a perfect square is. A perfect square is ...

One way to approach this is to count the numbers that are divisible by each of these perfect squares ...

Let's list out the perfect squares less than or equal to 50: ...

Next, I'll find the numbers in the set $\{1, 2, 3, \dots, 50\}$ that are divisible by each ...

Starting with 4:

Prompt:

How many of the numbers from the set $\{1, 2, 3, \dots, 50\}$ have a perfect square factor other than one?

<|begin_of_thought|>

So I have this problem: "How many of the numbers from ...

First, I need to understand what a perfect square is. A perfect square is ...

One way to approach this is to count the numbers that are divisible by each of these perfect squares ...

Let's list out the perfect squares less than or equal to 50: ...

Next, I'll find the numbers in the set $\{1, 2, 3, \dots, 50\}$ that are divisible by each ...

Starting with 4: ...

...

If I add all these up: $12 + 5 + 3 + 2 + 1 + 1 = 24$.

But wait, some numbers might be counted multiple times because they are divisible by more ...

So, I need to use the principle of inclusion-exclusion to avoid double-counting. ...

...

But wait, let's double-check this

...

Alternatively, maybe I should ...

...

Wait, that's 20 numbers. But earlier I had 23. There must be a mistake in my earlier reasoning.

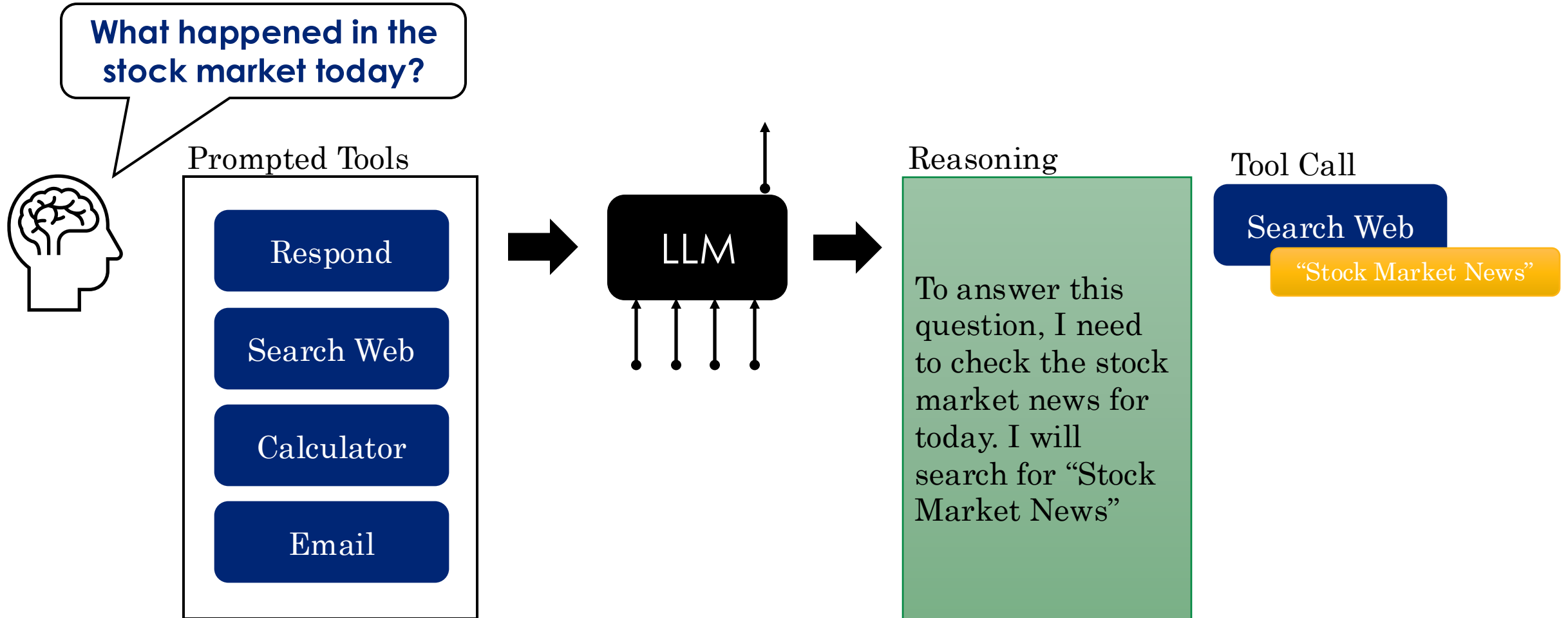
...

So, the final answer is 19.<|end_of_thought|> <|begin_of_solution|> ...

450 lines (3000 words)
of “Reasoning”

Agentic Systems

Enabling LLMs to *explore the world*



Agentic Systems

Enabling LLMs to *explore the world*

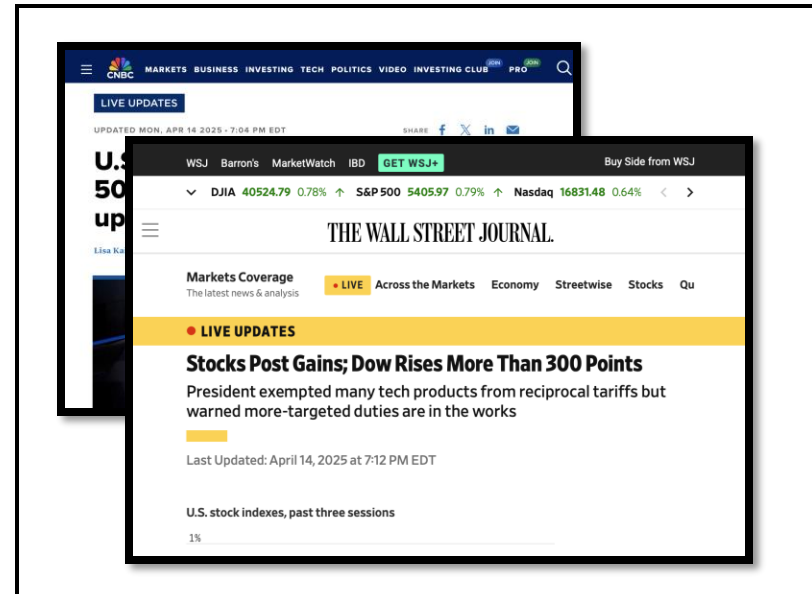
Tool Call

Search Web

“Stock Market News”



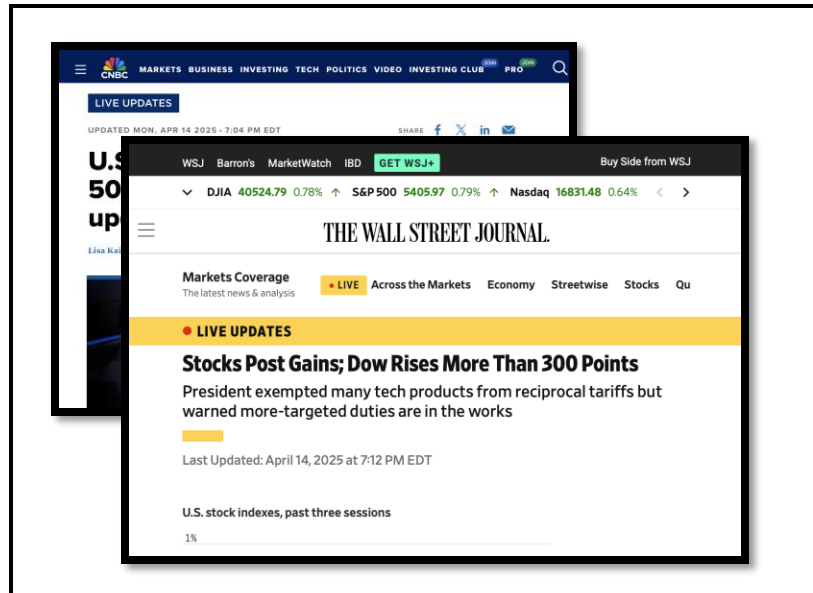
Tool Output



Agentic Systems

Enabling LLMs to *explore the world*

Tool Output



History

What happened in the stock market today?

To answer this question, I need to check the stock market news for today. I will search for "Stock Market News"

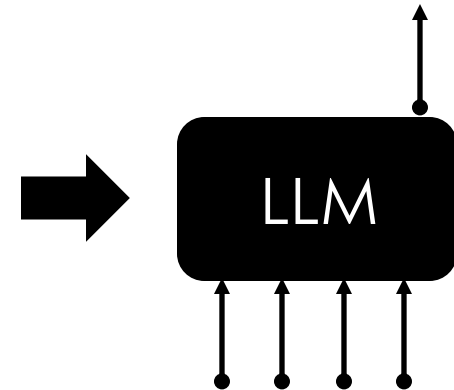
Prompted Tools

Respond

Search Web

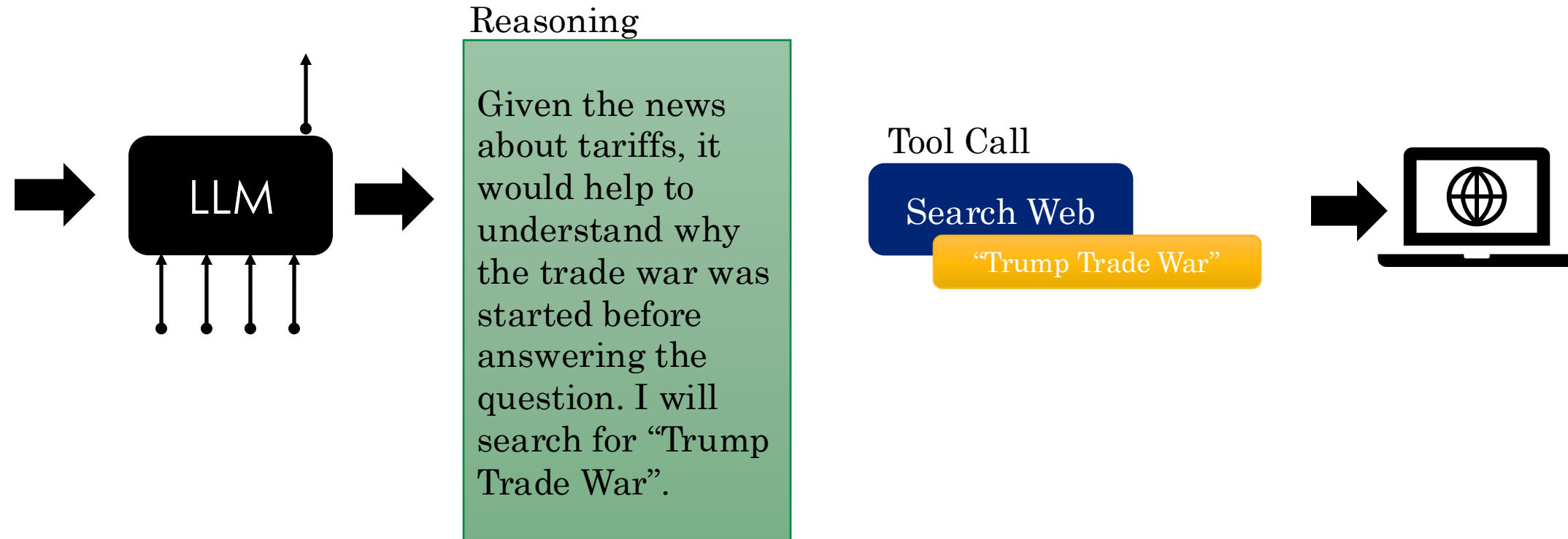
Calculator

Email



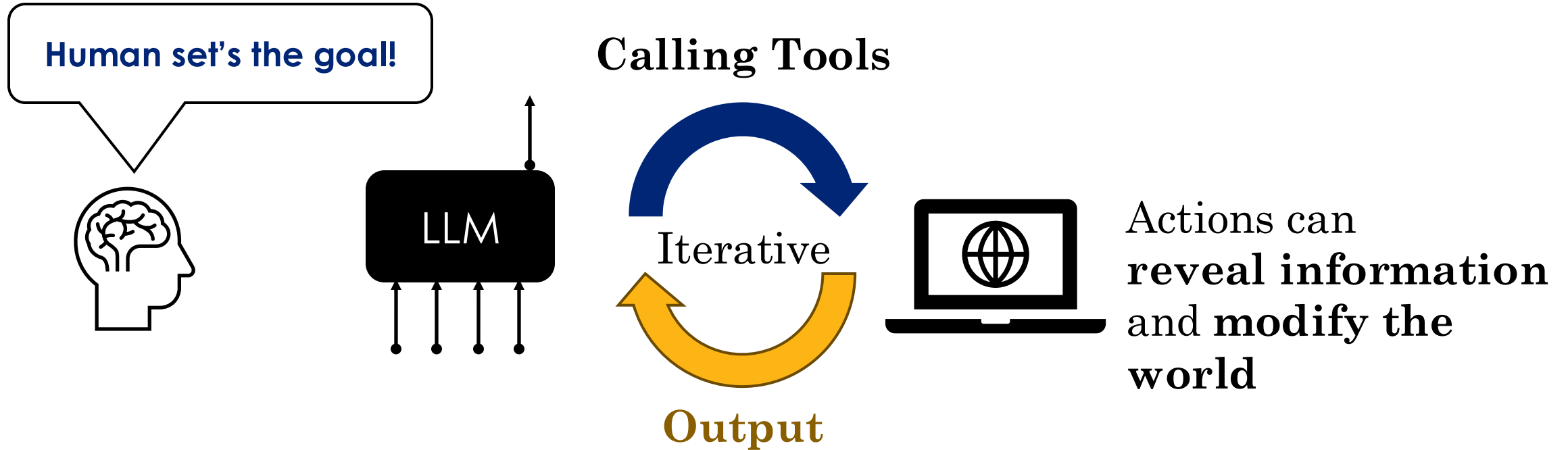
Agentic Systems

Enabling LLMs to *explore the world*



Agentic Systems

Enabling LLMs to *explore the world*



Reason-Act (ReAct) Agents: A lot of excitement about what these new systems can do.

Lecture 23

LLM Training and Applications

Reading: *Chapter 10* in Bishop Deep Learning Textbook